

Autonome Eingebettete Systeme (AEiSy)

Level 2 – C in eingebetteten Systemen

- 1 Embedded Systeme
- 2 Werkzeuge für die embedded SW Entwicklung
- 3 Embedded C
- 4 Embedded Software
- 5 Tipps und Tricks

Fakultät Elektrotechnik und Informatik

Level 2: C in eingebetteten Systemen

Lernziele:

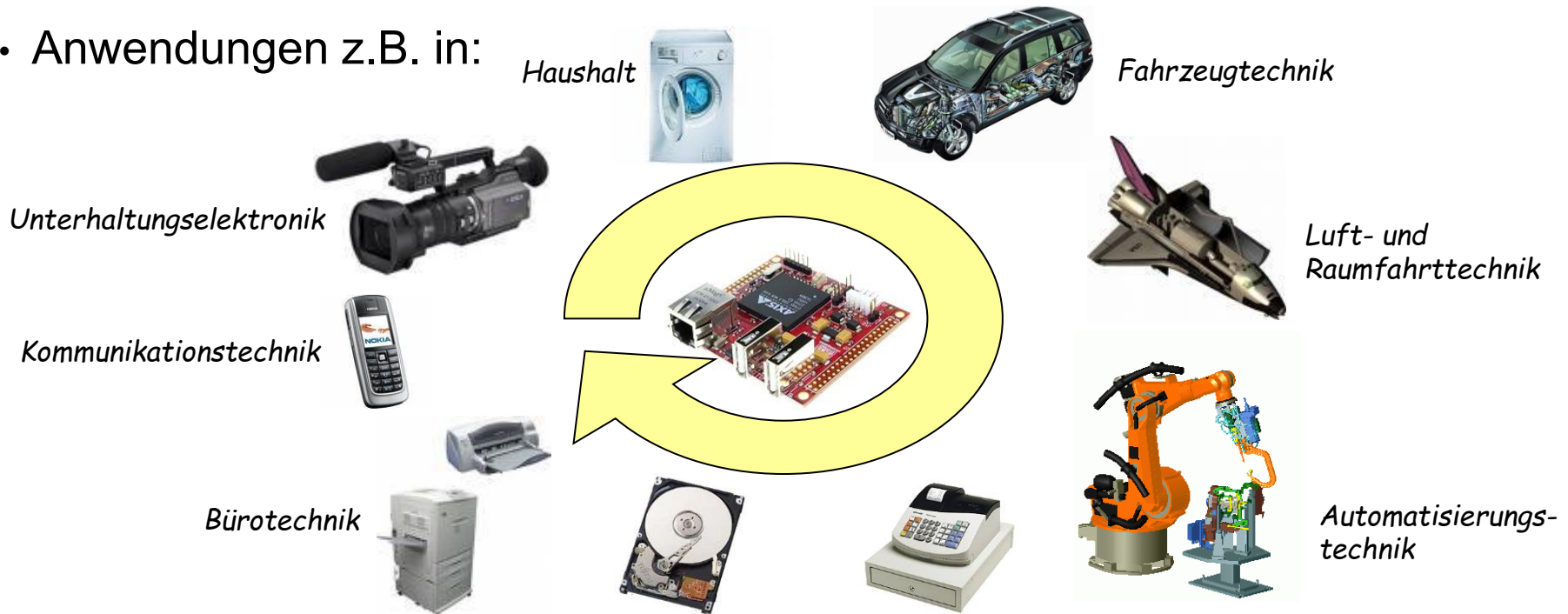
- Wiederholung / Vertiefung der wichtigsten C-Elemente/Strukturen
- Besonderheiten von C in Embedded Software
- Tipps und Tricks

Nicht: C – Grundlagenvorlesung !!

1. Embedded Systeme

- Anwendungsfeld für **Mikrocontroller**
- „**eingebettet**“ (embedded) in ein technisches Umfeld zur **Steuerung, Regelung** oder **Überwachung** dieses Umfeldes

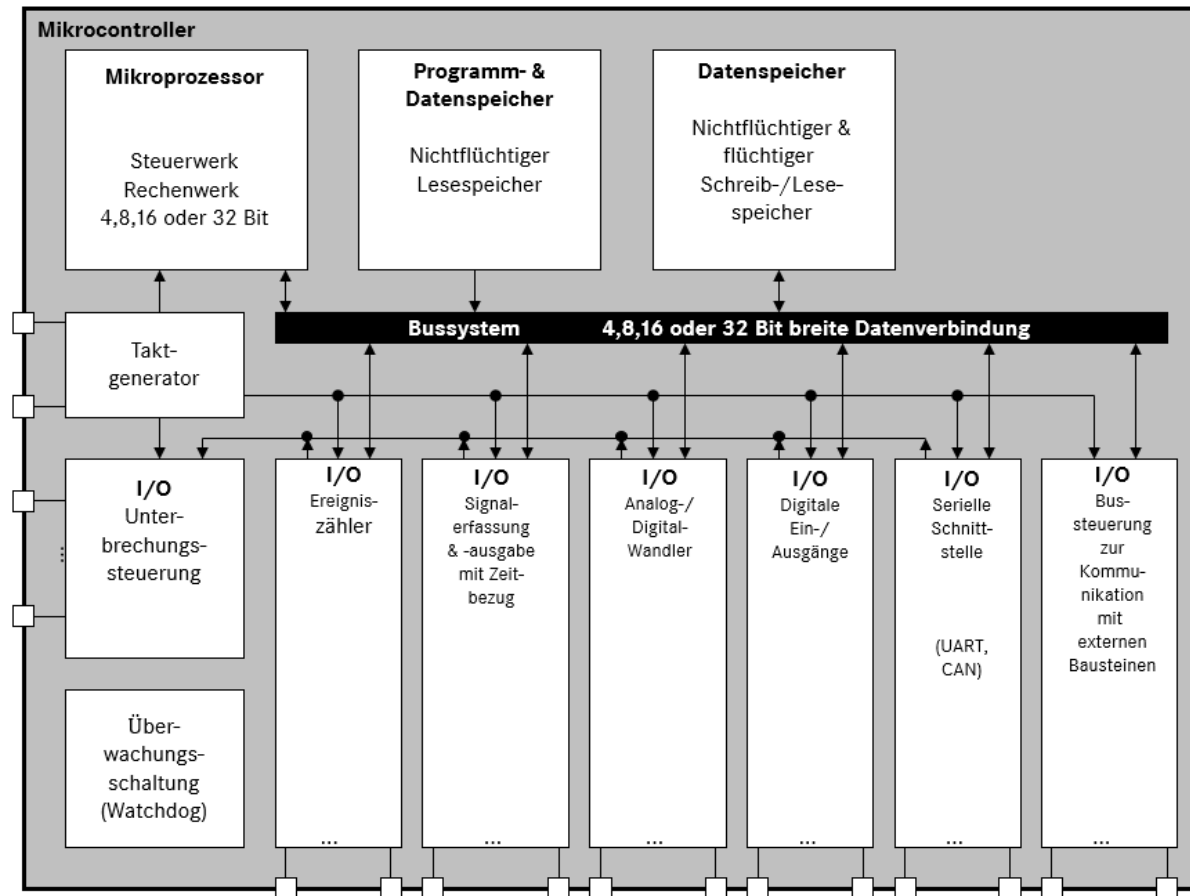
- Anwendungen z.B. in:



a) Mikrocontroller

= Mikrorechner, jedoch auf einen Chip integriert

- für eine Anwendung hin optimierte Peripherie
- eine Vielzahl von μC - Derivaten mit teilweise identischem Prozessorkern
- oft viel längerer Lifecycle als bei *general purpose CPUs*



Quelle: Jörg Schäuffele, Thomas: Automotive Software EngineeringVieweg & Teubner Verlag

weitere Begriffe

- **Signalprozessoren (DSP)**

- μC mit zusätzlichen Co-Prozessor zur digitalen Signalverarbeitung (z.B. für digitale Filter, Bild- und Audioverarbeitung)
- MAC = multiply & accumulate Befehl: $a \leftarrow a + (b \cdot c)$

- **SoC** = Systems-on-Chip (Ein-Chip-System)

- μC alleine nicht einsetzbar $\Rightarrow$ zusätzliche HW notwendig
z.B. Graphik, Audio, Co-Prozessoren, Schnittstellen (Ethernet/CAN)
- Übergang von μC zu SoC sind fließend

Beispiel: Raspberry Pi 4

(BCM2711, 1500 MHz quad-core ARM Cortex-A72 CPU)

- **FPGA** = Field Programmable Gate Array

- rekonfigurierbare Hardware = Programmierung von logischen Schaltungen
- meist Geschwindigkeitsvorteil gegenüber SW-Lösung

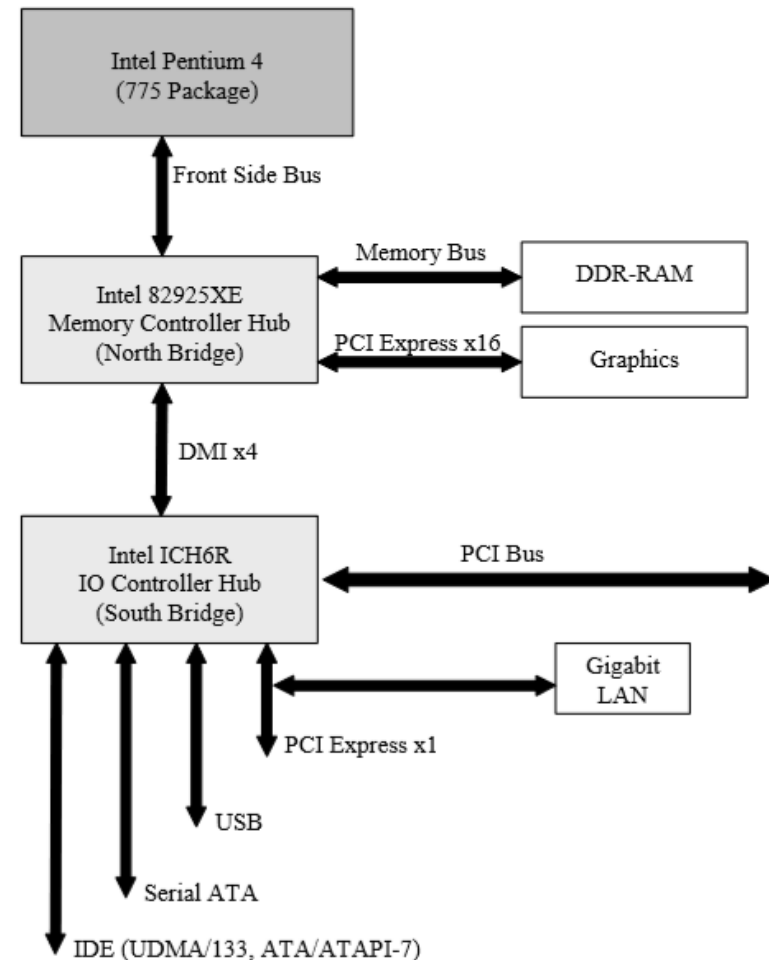
b) Abgrenzung zu PC - Systemen

- Personal Computer (PC) = Mikrorechner
- typisch:
Aufbau aller wesentlichen Komponenten auf dem **Motherboard** (Chipsatz).

viele **Schnittstellen**

Erweiterbarkeit über Steckkarten
(PCI)

⇒ für embedded Anwendungen meist
zu groß, teuer



d) Anforderungen an ein Embedded System

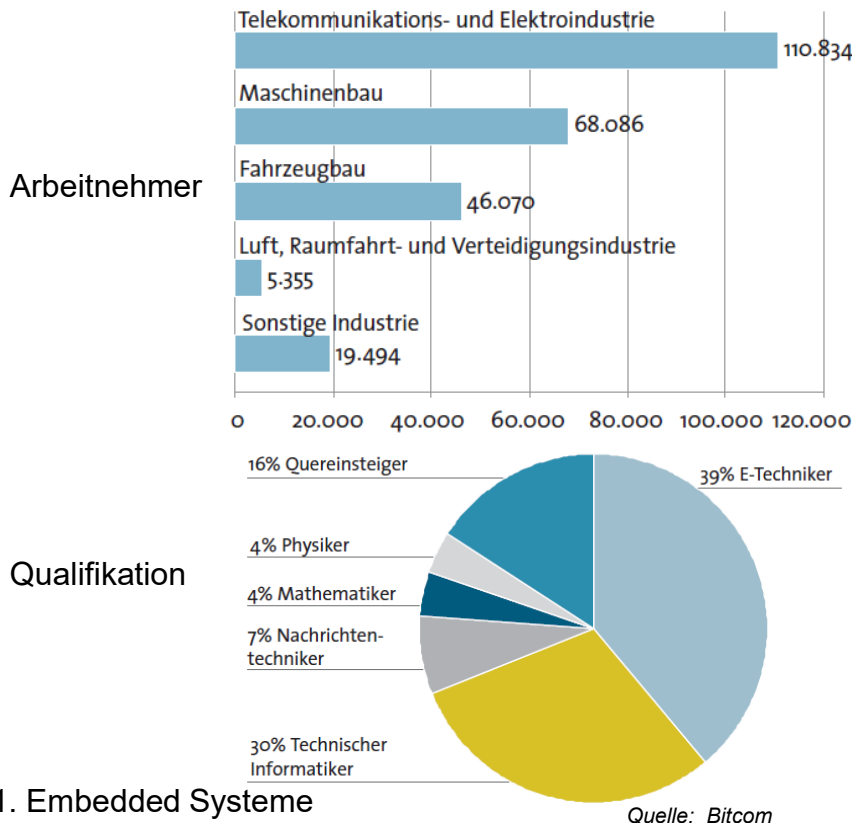
- Einsatz unter **rauen und wechselnden Umgebungsbedingungen** in Bezug auf
 - Temperaturbereich, Feuchtigkeit, Erschütterungen
 - hohe Anforderungen an elektromagnetische Verträglichkeit (EMV)⇒ Unterscheidung: *Consumer, Industry, Automotive, Military/Aerospace*
- hohe Anforderungen an die **Zuverlässigkeit** und **Verfügbarkeit**
⇒ oft ein Qualitätsmerkmal („Liegenbleiber“)
- hohe Anforderungen an (Funktionale) **Sicherheit**
⇒ Risikominimierung und Schadensbegrenzung (Bremsen, Airbag, ..)

ASIL A,B,C,D (ISO26262) SIL 1,2,3,4 (ISO 61508)
- vergleichsweise lange **Produktlebenszyklen**
⇒ Unterscheidung: Consumer (Handy,PC) ↔ Automobil (Ersatzteile)

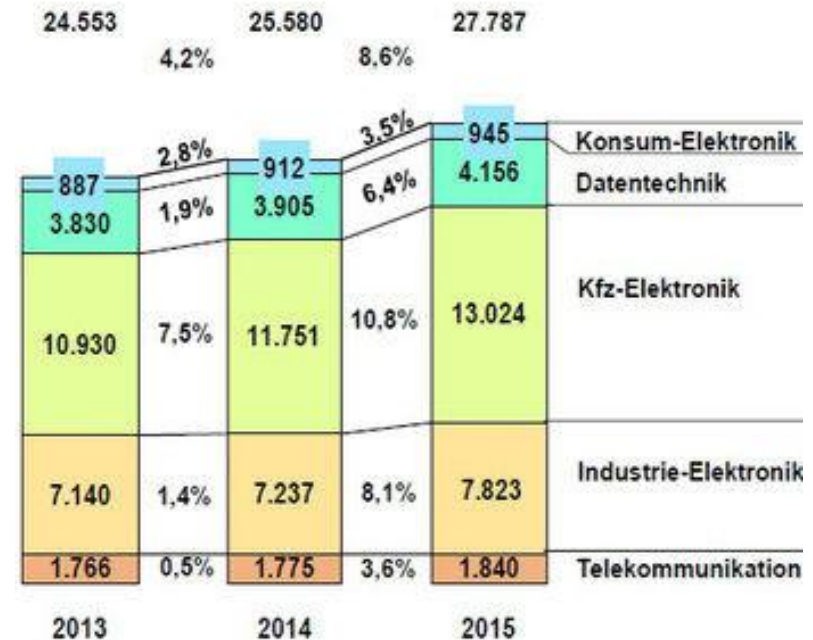
e) Wirtschaftliche Bedeutung von Embedded Systemen

- Fast 90% aller elektronischen Bauteile werden in eingebetteten Systemen verbaut.
- Embedded Systeme stellen einen sehr großen Markt dar:

Arbeitsmarkt:

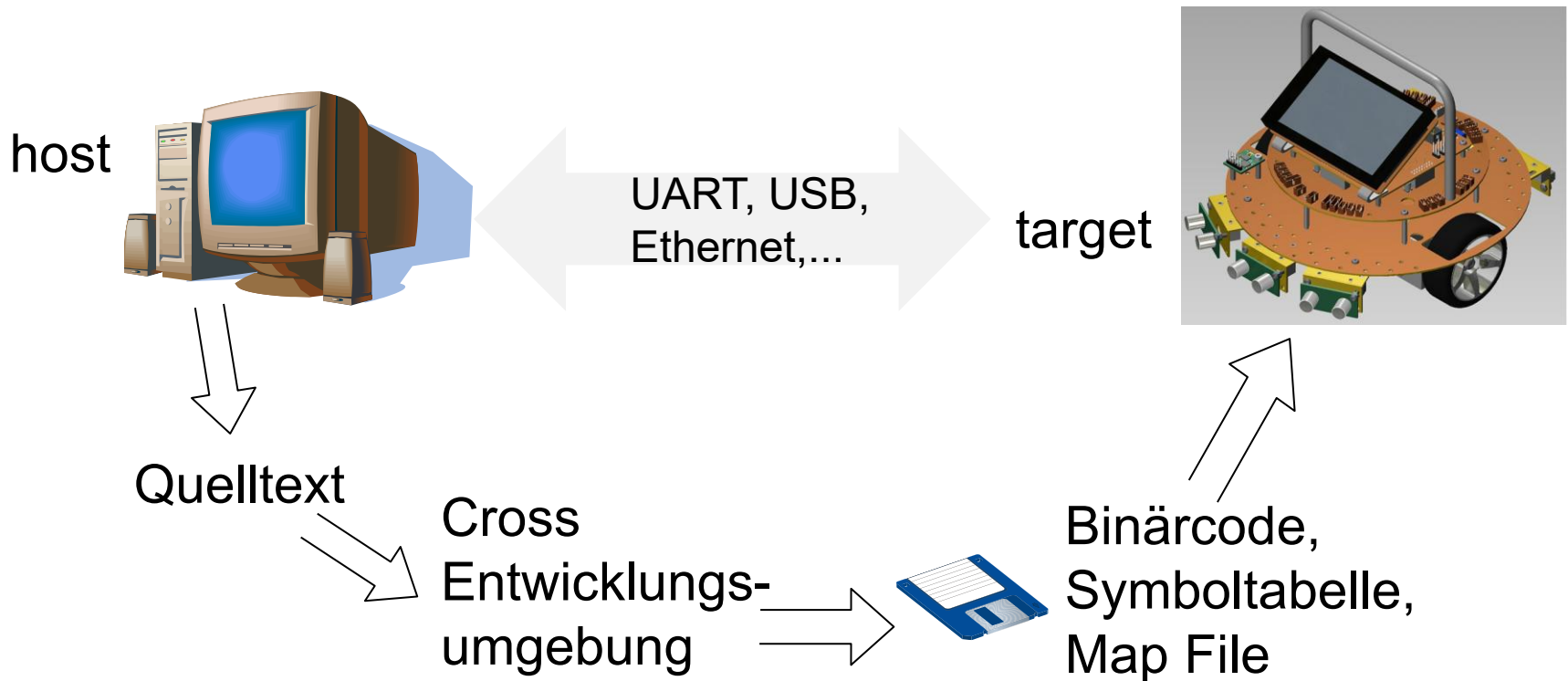


Hersteller von elektronischen Baugruppen:



2. Werkzeuge zur Embedded SW-Entwicklung

- Cross – Entwicklungsumgebung:
⇒ Entwicklungssystem $\neq$ Zielsystem



- Bemerkungen:

- Die Entwicklung der **embedded SW** erfolgt im Allgemeinen auf einem PC (**Host**).

Hostplattformen: Windows/Linux



- Der passende Binärcode wird mit Hilfe eines **Cross Compilers/ Cross Assemblers** auf dem Host erzeugt.

Binärcode in HEX - Format

- Die übersetzte embedded SW wird auf die Zielhardware (**Target**) geladen und dort ausgeführt.
- Die **IDE** (*integrated development environment*) umfasst die gesamte Toolchain (Editor, Compiler, Linker).

```
:0200000020000FC
:100000000C9420000C946B000C946F000C946F0007
:100010000C946F000C946F000C946F000C946F00A4
:100020000C946F000C946F000C946F000C946F0094
:100030000C946D000C946F000C946F000C946F0086
:100040000FEF0DBF00E10EBF0FEF04B90FEF05B9C1
:1000500051E060E002E00093690001E00DBB81E047
:1000600000E0009380000DE00093810000E0009329
:1000700082000DE30093890009E00093880002E00C
:1000800000936F0090E07894052F009505B97030CB
:1000900029F4550F503031F451E004C056955030DA
:1000A00009F450E89030F1F390E0EECF4AE038ECFC
:1000B000000000000000000000003A953030C9F7000051
:1000C0004A954030A1F700001A951F3F79F72A950D
:1000D0002F3F61F708957827189591E018951895A6
:00000001FF
```

Sprechen Sie C?

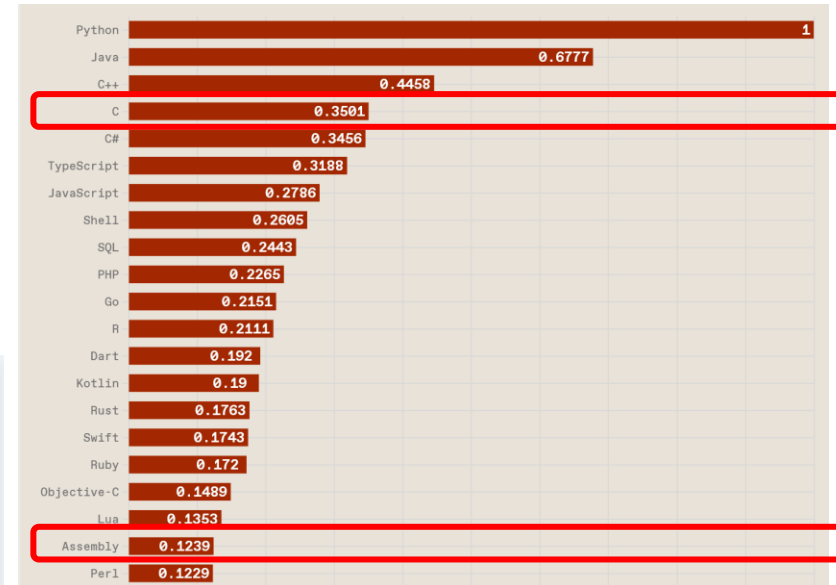
- **C ist Industriestandard**,
vor allem in Embedded Systemen

→ sehr gut geeignet für
HW-nahe Programmierung!

Mar 2026	Mar 2025	Change	Programming Language	Ratings	Change
1	1		Python	21.25%	-2.59%
2	4	▲	C	11.55%	+2.02%
3	2	▼	C++	8.18%	-2.90%
4	3	▼	Java	7.99%	-2.37%
5	5		C#	6.36%	+1.49%
6	6		JavaScript	3.45%	-0.01%
7	9	▲	Visual Basic	2.50%	-0.02%
8	8		SQL	2.00%	-0.57%
9	16	▲	R	1.88%	+0.94%
10	10		Delphi/Object Pascal	1.80%	-0.36%
11	24	▲	Perl	1.75%	+1.05%
12	12		Scratch	1.63%	-0.03%
13	11	▼	Fortran	1.45%	-0.25%
14	14		Rust	1.31%	+0.09%
15	15		MATLAB	1.29%	+0.31%
16	7	▼	Go	1.29%	-1.49%
17	17		Assembly language	1.29%	+0.42%

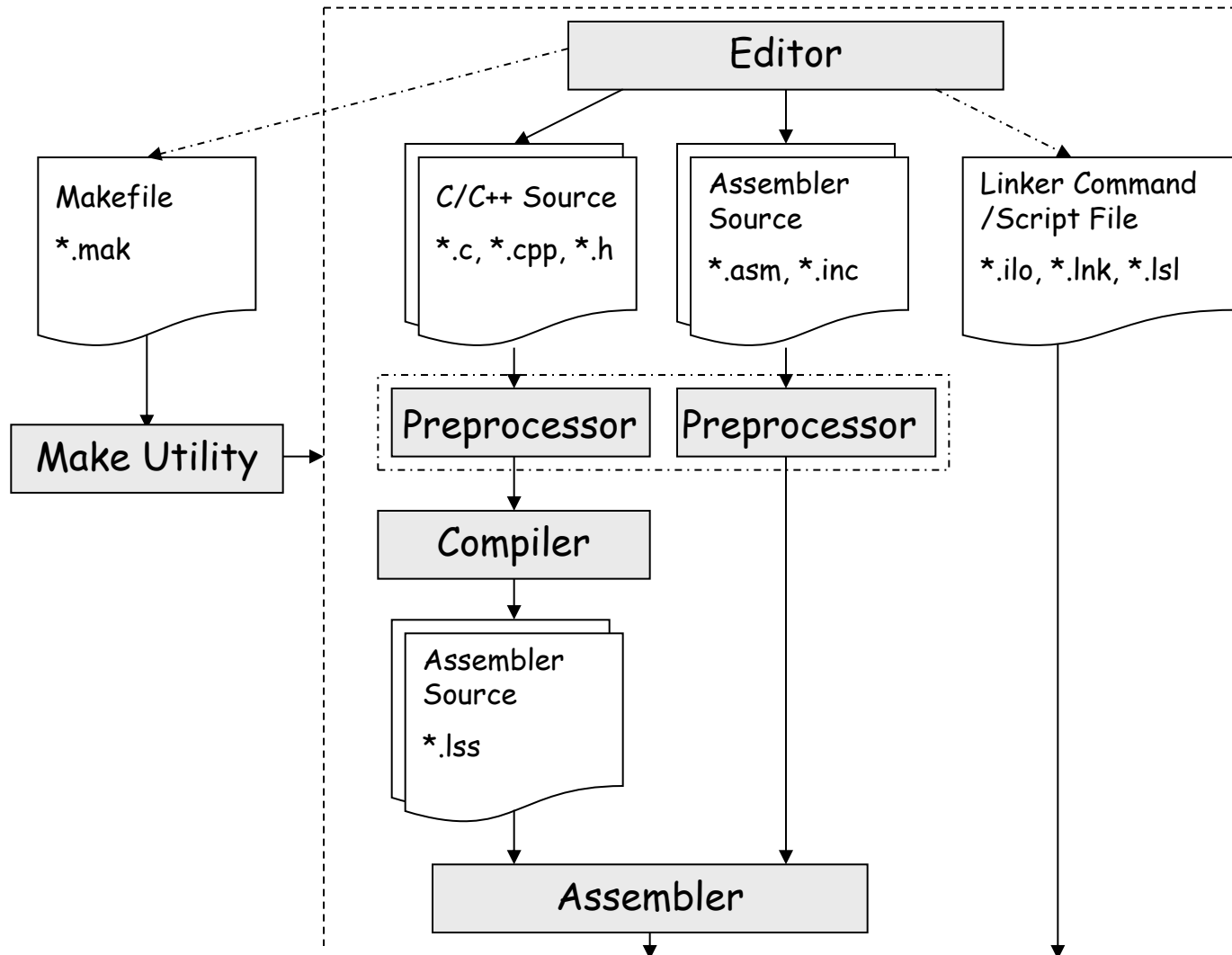
Quelle: spectrum.ieee.org

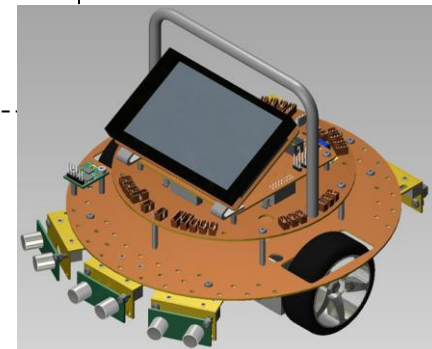
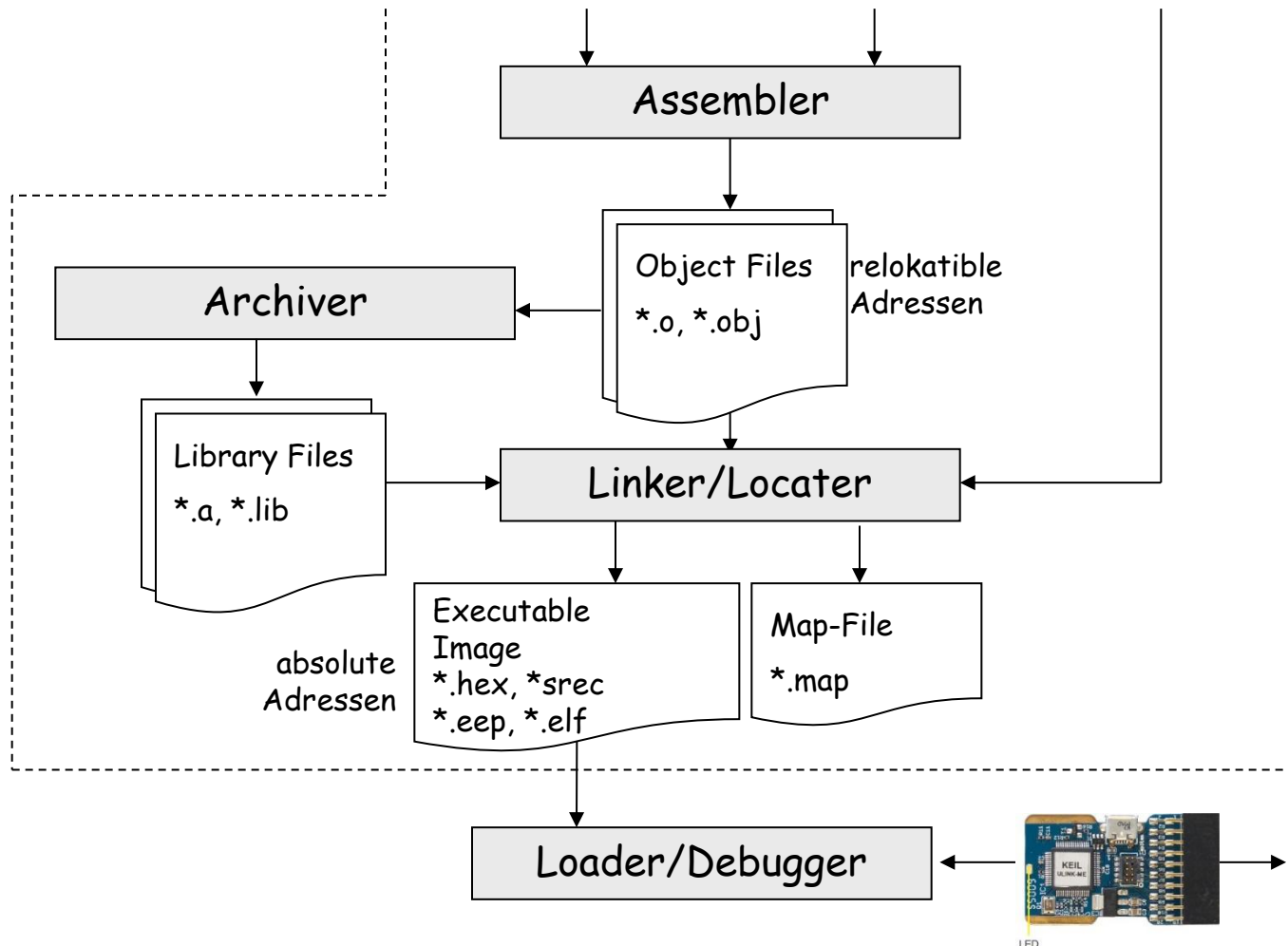
2026



- Motivation für Assembler:
 - effiziente Ausnutzung der Ressourcen des μC
 - Didaktik!

- Entwurfsablauf, Toolchain:

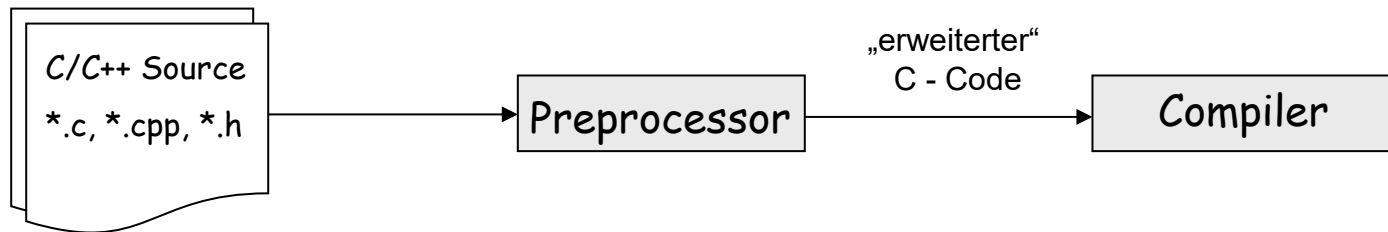




3. Embedded C

3.1 Präprozessor und Macros

- Der Präprozessor wird bei jeder Quellcode – Datei (*.c / *.h) vor dem eigentlichen Compilervorgang ausgeführt.



- Präprozessoranweisungen (*Direktiven*) sind mit einem # gekennzeichnet.
- Beispiele:

```
#include <stm32g474xx.h>
```

```
#define PI (3.14f)
```

```
#error "No version defined"
```

Die `#define` Direktive

- einfache Textersetzung
- Anwendung: z.B. Definition von Konstanten (und Symbolen):

```
#define MAXVALUE 100
int temp = 200;
if( temp > MAXVALUE) // if ( temp > 100 )
{
    temp = MAXVALUE; // temp = 100;
}
```

Nachteil: Der Präprozessor führt keine Typüberprüfung durch!
Der Typ leitet sich vom Literal ab:

```
#define MAXVALUE 100 // -> integer (16 Bit)
#define MAXVALUE 100L // -> long integer (32 Bit)
#define MAXVALUE 100UL // -> unsigned long integer (32 Bit)
#define MAXVALUE 100.0f // -> float (einfache Genauigkeit, 32Bit)
#define MAXVALUE 'd' // -> character (8Bit)
```

oder: expliziter Cast

```
#define MAXVALUE (int)100
```

Der Präprozessor als Taschenrechner

- Der Präprozessor wertet bereits alle Ausdrücke mit konstantem Wert aus.
- Beispiel: Anzahl der Sekunden eines Jahres

```
#define SECONDS_PER_YEAR (60UL * 60UL * 24UL * 365UL)
```

- Bemerkungen:
- kein ; am Ende
 - Klammern setzen
 - einfache Literale werden als Integer gewertet
 - ⇒ Gefahr des Überlaufs bei 8/16 Bit Architekturen!
 - ⇒ Suffix **L** (oder **UL**) für **long** (oder **unsigned long**) Werte
 - funktioniert auch mit **float/double**:

```
#define AREA (2.0 * 3.1415 * 10.0 * 10.0)
```


Makros

- Die Textersetzungsfunktion der **#define** Direktive kann auch dazu verwendet werden, um Werte und Ausdrücke erst während der Laufzeit zu bewerten.
- Beispiele:
 - Addition von zwei Zahlen

```
#define ADD(A,B)      (A + B)
sum = ADD(3,4);
```

- die kleinere von zwei gegebenen Zahlen

```
#define MIN(A,B) ((A) <= (B) ? (A) : (B))
least = MIN(3,4);
```

- mehrzeilige Macros (*Escape Zeichen*)

```
#define LIMIT(x) { \
    if ( x > 9 ) \
        x = 9; \
    else if ( x < 0 ) \
        x = 0; \
}
```

Geht, aber lieber vermeiden!

- + kein Funktionsaufruf, typunabhängig
- Typcheck, Debugging, ...

```
static inline int limit(int x)
{
    return (x > 9) ? 9 : (x < 0) ? 0 : x;
}
```

problematische Makros

- Beispiel 1:

```
#define Calculation(A,B) A = B * B + B
...
unsigned long int a=3,b=3;
...
Calculation(a,b);           // a = ?
Calculation(a,b+2);         // a = ?
Calculation(a,++b);         // a = ?
```

- Beispiel 2:

```
#define Calc(A,B) {A=A+1; B=B+2;}
...
int a=3,b=3,i=1;
...
if(i == 0)
    Calc(a,b); // -> { a=a+1; b=b+2; };
else
    Calc(b,a); // -> { b=b+1; a=a+2; };
```

#ifdef / #if / #else / #endif

- Bedingte Übersetzung:

```
#define PFERD 4711
#ifdef PFERD
printf("Pferd.\n");
#else
printf("Kein Pferd.\n");
#endif
```

→
wird zu

```
printf("Pferd.\n");
```

- Fallunterscheidungen

```
#define PFERD 4711
#if PFERD == 815
printf("Pferd.\n");
#else
printf("Kein Pferd.\n");
#endif
```

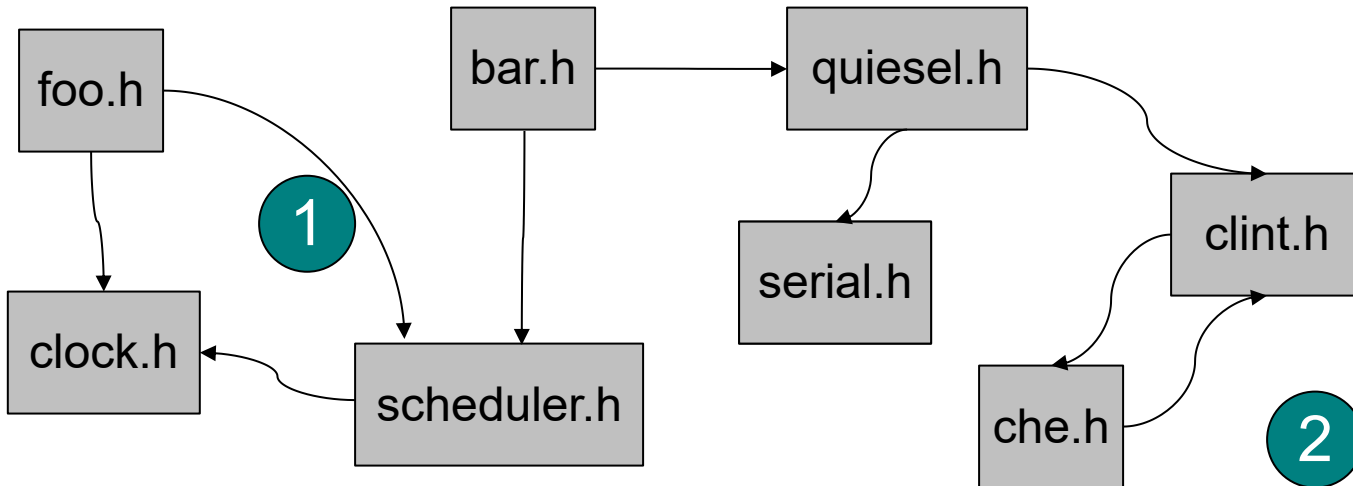
→
wird zu

```
printf("kein Pferd.\n");
```

Include Wächter

`#include <file>` Präprozessor kopiert *file* textuell an diese Stelle

- Szenario: Viele Headerfiles, gegenseitiges Einbinden



- Inhalte der Headerfiles wiederholen sich (bei ①)
- Endlosrekursion (bei ②)

- Lösung: **Include-Wächter**

`<whatever>.h`

```
#ifndef __WHATEVER_H__
#define __WHATEVER_H__
/* Inhalt des Headers */
#endif
```

Stringification

- Die Argumente eines Makros werden als String gewertet.
- Beispiel:

```
#define WARNING (EXP)    printf("Warning:" #EXP "\n")
...
WARNING (E123) ;
// ->    printf("Warning:" "E123" "\n")
// ->    printf("Warning: E123 \n")
```

→ #: *mach einen String daraus*

Concatenation

- Die Argumente eines Makros werden zu einem Token verknüpft.

- Beispiel:

```
#define Button_bit    1
#define Button_port   PORTA

#define SetPin(pin, value) \
    { \
        if ( value ) \
            pin ## _port |= 1 << pin ## _bit; \
        else \
            pin ## _port &= ~(1 << pin ## _bit); \
    }

...
SetPin( Button, 1);
```

```
typedef struct {
    volatile uint32_t *port;
    uint8_t bit;
} Pin_t;

static const Pin_t Button = { &GPIOA->ODR, 1 };

static inline void SetPin(Pin_t p, int value) {
    if (value) *p.port |= (1u << p.bit);
    else      *p.port &= ~(1u << p.bit);
}
```

```
{
    if ( 1 )
        PORTA |= 1 << 1;
    else
        PORTA &= ~(1 << 1);
};
```

Die **#error** Direktive

- Die **#error** Direktive gibt den folgenden String im Ausgabefenster des Compilers aus und generiert einen Compilerfehler.
- Beispiel:

```
#ifndef VERSION
    #error "No version defined"
#endif
```

→ Anwendung: z.B. Test, ob wichtige Symbole definiert wurden

- Hinweis: **#warning** gibt ebenfalls einen String im Ausgabefenster aus,
jedoch wird damit kein Fehler generiert.

3.2 Variablen und Datentypen

3.2.1 Variablenvereinbarung

- **Variable** entspricht einem reserviertem **Speicherbereich** (Adresse) im Arbeitsspeicher eines Rechners, der durch einen **Bezeichner** (symbolische Adresse) und **Datentyp** (Interpretation) eindeutig repräsentiert wird.
- Syntax: Variablendefinition ($\leftrightarrow$ Variablendeklaration ?)

[Speicherklasse] Datentyp Variablenname [, Variablenname];

Beispiele:

- (einfache) Standardtypen:

char	Zeichen (ASCII-Code), 8 Bit
int	ganze Zahl, (32 Bit*)
float	Gleitkommazahl (32 Bit)
double	doppelt genaue Gleitkommazahl (64 Bit)
void	ohne Wert

```
int    nummer;
int    array[10];
long   zahl;
double steigung, xAchse, yAchse;
```

char	8 Bit
short	16 Bit
int	32 Bit
long	32 Bit
long long	64 Bit
Pointer	32 Bit

Konkret definiert
durch (Embedded)
Application Binary
Interface: (E)ABI

- **Modifikatoren:**

signed, unsigned,
(short, long)

Syntax:

*[Speicherklasse] **Datentyp** Variablenname;*

- Datentyp **boolean** gibt es nicht (bzw. erst ab C99: **_Bool**):

- boolesche Ausdrücke evaluieren zu 0 (falsch) oder zu 1 (wahr)
- ganze Zahlen können wie boolesche Variablen verwendet werden

```
printf("%d", 4711 > 42 );    /* gibt 1 aus */
```

- auch konstante Ausdrücke (Literele) haben eine Datentyp

```
#define MAXVALUE 100    // -> integer (32 Bit)
#define MAXVALUE 100L   // -> long integer (32 Bit)
#define MAXVALUE 100UL   // -> unsigned long integer (32 Bit)
#define MAXVALUE 100.0f   // -> float (einfache Genauigkeit, 32Bit)
#define MAXVALUE 'd'    // -> character (8Bit)
```

C-Standard (ISO/IEC 9899), Abschnitt 6.7.2 Type specifiers:

```
void
char
signed char
unsigned char
short, signed short, short int, signed short int
unsigned short, unsigned short int
int, signed, signed int
unsigned, unsigned int
long, signed long, long int, signed long int
unsigned long, unsigned long int
long long, signed long long, long long int, signed long long int
unsigned long long, unsigned long long int
float
double
long double
```

Das ist alles!

Die Aufzählung pro Zeile ist äquivalent.

Besonderheit: char steht allein, compilerabhängig ...

Rang bzw. Integer Promotion:

```
char, short, ... -> int -> unsigned int -> long -> unsigned long -> long long
-> unsigned long long -> float -> double -> long double
```

- **Problem der Standardtypen:**

Im C-Standard sind die Bitbreiten der Datentypen nicht definiert!

Beispiel:

```
int    x = 10;    // Wieviel Bits hat x?
```

Endlosschleife,
wenn unsigned int
16 bit lang ist!

```
unsigned int counter = 0;  
while (1) {  
    counter++;  
    ... // anderer Code  
    if (counter > 65535) break;  
}
```

⇒ Vermeidung der Standardtypen! → *stdint.h*:

⇒ bessere Portierbarkeit!

z.B. MISRA 6.3:

„typedefs that indicate size and signedness
should be used in place of the basic types.“

```
typedef signed char      int8_t;  
typedef signed int       int16_t;  
typedef signed long int  int32_t;  
typedef signed long long int int64_t;  
typedef unsigned char    uint8_t;  
typedef unsigned int      uint16_t;  
typedef unsigned long int uint32_t;  
typedef unsigned long long int uint64_t;
```

<i>[Speicherklasse]</i> Datentyp <i>Variablenname;</i>

- **Speicherklassen:**

`const, static, volatile, extern, register`

- Speicherklassen beeinflussen die Lebensdauer, den Gültigkeitsbereich (Sichtbarkeit, Scope) oder auch den Speicherplatz von Variablen.

- (1) `const`: Der Wert der Variable kann nicht verändert werden.
⇒ „*read-only*“

Hinweis: Der Speicherplatz muss nicht zwingend im Flash-Speicher (non-volatile memory) liegen.

- (2) `static`: Die *Lebensdauer* der Variable ist unbegrenzt.

auch: Möglichkeit des Zugriffsschutzes!

- lokale Variable: Der Wert der lokalen Variable bleibt für den nächsten Funktionsaufruf erhalten.
⇒ Speicherplatz im RAM / Lebensdauer
 - globale Variable: Der *Scope* der Variable wird eingeschränkt auf die C-Datei.

<code>[Speicherklasse]</code> <i>Datentyp</i> <i>Variablenname</i> ;

(3) `volatile`:

zeigt dem Compiler an, dass sich der Wert der Variablen verändern kann

z.B. durch

- Hardware (→ I/O Register),
- Interrupts,
- nebenläufige Tasks (*reentrant*) in einem RTOS

⇒ vor jedem lesenden Zugriff wird der Wert aktualisiert

⇒ werden vom Compiler nicht „weg optimiert“

(4) `register`: **Allokierung** der Variable in einem Arbeitsregister

⇒ nicht adressierbar (&-Operator ist nicht erlaubt)

⇒ nur bei lokalen Variablen (oder Parameter)

⇒ greift in die Codeerzeugung des Compilers ein

(5) `extern`: `extern int counter;`

Definition von `counter` in einer anderen C – Datei

⇒ keine Reservierung von Speicherplatz

3.2.2 Der „volatile“ Typ-Modifikator

• Was ist das Problem?

C-Compiler optimiert Code (-> Geschwindigkeit, Größe)

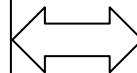
- Grundannahme: Alle Schreib-/Lesevorgänge auf Variablen werden von Compiler selbst erzeugt.
- Optimierung entfernt „überflüssige“ Zugriffe auf Variablen

Beispiel:

```
unsigned int* ptr = (unsigned int*) 0x1234;  
  
while (*ptr == 0); // Wait for register to become non-zero  
// Do something else.
```

ohne Optimierung

	MOVW	r3, #0x1234
	STR	r3, [SP]
	NOP	
loop:	LDR	r3, [SP]
	LDR	r3, [r3]
	CMP	r3, #0x00
	BEQ	loop



mit Optimierung

	MOVW	r3, #0x1234
	LDR	r2, [r3]
loop:	CMP	r2, #0
	BEQ	loop

- **Was macht die Optimierung?**

Optimierung nimmt an, dass keine Schreibzugriffe auf Speicher zwischen zweitem „ldr“ und „beq“ erfolgen

→ Damit muss Wert nur einmal aus *ptr in r3 geladen werden!

ABER: Speicher kann *asynchron* verändert werden

- Parallele Prozesse mit gemeinsamem Speicher
- Interrupt-Behandlungsfunktionen (z.B. für Gerätetreiber)
- „Memory mapped“ Hardware-Register, die externe Daten bereitstellen

⇒ r2 in jedem Schleifendurchlauf (unabhängig von der Optimierungsstufe!) neu laden!

Lösung:

volatile

```
volatile unsigned int* ptr = (unsigned int*) 0x1234;
```

	MOVW	r3, #0x1234
loop:	LDR	r2, [r3]
	CMP	r2, #0
	BEQ	loop

mit Optimierung

3.2.2 Der „static“ Typ-Modifikator

```
#include <stdio.h>

int zaehler(void) {
    static int n = 0;    // Initialisierung nur einmal beim Programmstart
    return ++n;
}

int main(void) {
    printf("%d\n", zaehler()); // 1
    printf("%d\n", zaehler()); // 2
    printf("%d\n", zaehler()); // 3
    return 0;
}
```


3.2.2 Der „register“ Typ-Modifikator

```
#include <stdint.h>

void delay_loop(uint32_t count) {
    register uint32_t i;
    for (i = 0; i < count; i++) {
        __asm volatile("nop");
    }
}
```

wird vom Compiler ggf. ignoriert ...

```
register uint32_t tick __asm("r4");
```

Erzwingt Register R4, für inline-Assembly hilfreich.

Dass R4 belegt ist, erkennt der Compiler aber nicht und nutzt es selbst!

3.2.3 *typedef*

- Definition eines **neuen Namens** für existierenden Typ
- Syntax: wie beim Anlegen einer Variablen, **typedef** davor

```
typedef int Length; /* diese Zeile belegt keinen Speicher! */  
  
Length len, max_length;  
void set_length(Length l) { ... }
```

- **Abstraktionsmittel:** der tatsächlich verwendete Typ wird versteckt, kann leicht ausgetauscht werden (prominente Beispiele: pid_t, FILE)
- **Selbstdokumentation:** einfacher Name besser verständlich/lesbar als komplexer Pointer auf Struktur

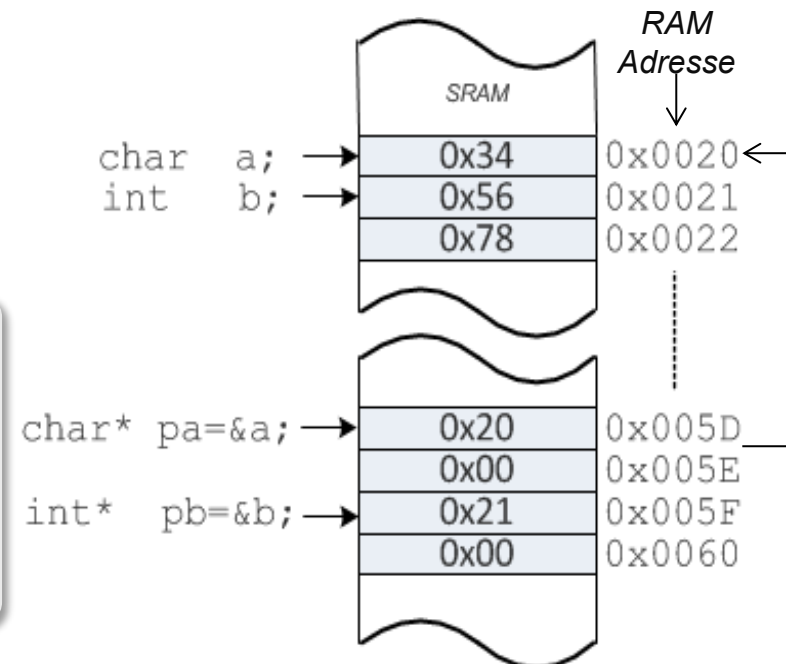
```
typedef struct listelement *LEPtr;  
LEPtr elem1, elem2;  
LEPtr find_elem(LEPtr firstelem, int searchnum) { ... }
```

3.2.4 Zeiger (Pointer)

- **Zeiger** (-variable, *Pointer*) sind **Variablen**, deren **Wert** als **Adresse** auf eine andere Variable (oder Funktion) interpretiert wird.
- Zeiger sind also eine Referenz auf ein Datenobjekt.
→ Sie „zeigen“ auf eine Variable.
- Syntax:

[Speicherklasse] **Datentyp** * *Zeigername*;
- Zugriff auf Variable über **Dereferenzierungsoperator** *
- **Adressoperator** &:
&x liefert die Adresse der Variablen x.

```
char a = '4';
int b = 0x7856;
char* pa = &a; /* zeigt auf Adresse von a */
int* pb = &b; /* zeigt auf Adresse von b */
int x = *pb; /* kopiert den Wert, worauf pb
              zeigt, nach x */
```



- Hinweis:

Die Bitbreite von Zeigervariablen hängt von der Architektur (Speichergröße) ab.

Beispiel: Cortex M3/4 : $\Rightarrow$ 32-Bit $\Rightarrow$ adressierbare Adressraum: 4 GB

- Verwendung von Zeigern:

- Zugriff auf Arrays
 - *Call by Reference*: Funktionen können ihre Argumente ändern
 - Dynamische Speicherverwaltung ⚡
 - Übergabe von großen Parameterobjekten an Funktionen ($\rightarrow$ struct)
 - Rückgabewert mehrerer Werte einer Funktion
 - Funktionszeiger ($\rightarrow$ später noch genauer)
- **ABER:**
 - Programmstruktur wird unübersichtlicher (welche Funktion kann auf welche Variablen zugreifen?)
 - häufigste Fehlerquelle in C-Programmen (z.B. Zeiger auf Zeiger ...)

- Bemerkungen zu Zeigervariablen:

- Zeigerarithmetik:

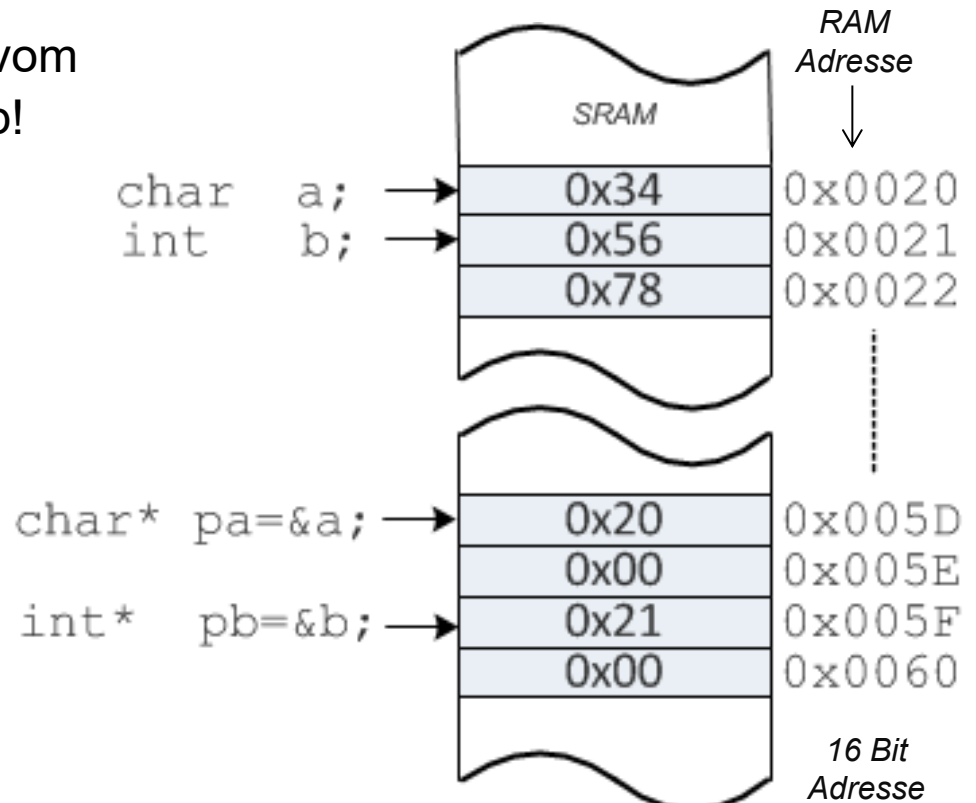
```
char c;
pa++;
c = *pa; // welchen Wert hat c?
```

Hinweis: Adresserhöhung hängt vom Datentyp des Zeigers ab!

- Speicherklasse `const`:

```
const char *pa;
pa = &a;
(*pa)++; // Fehler
pa++;    // kein Fehler
```

```
char * const pa = &a;
(*pa)++; // kein Fehler
pa++;    // Fehler
```

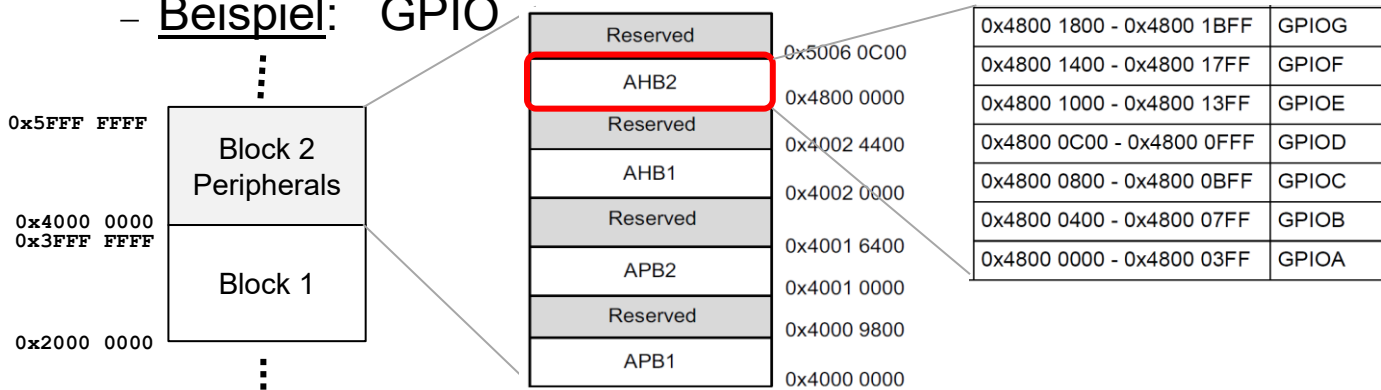


Normalerweise sind das 32 Bit Adressen!

• Anwendung von Zeigern in Embedded Software:

- I/O Register haben feste Adressen. $\Rightarrow$ Zugriff über feste Speicheradresse

– Beispiel: GPIO



```
typedef struct
{
    __IO uint32_t MODER;
    __IO uint32_t OTYPER;
    __IO uint32_t OSPEEDR;
    __IO uint32_t PUPDR;
    __IO uint32_t IDR;
    __IO uint32_t ODR;
    __IO uint32_t BSRR;
    __IO uint32_t LCKR;
    __IO uint32_t AFR[2];
    __IO uint32_t BRR;
} GPIO_TypeDef;
```

Die Register des **Ports A** beginnen bei Adresse **0x48000000**.

$\rightarrow$ Adresswert wird als Zeiger auf eine Struktur interpretiert:

```
#define GPIOA ((GPIO_TypeDef*) 0x48000000)
```

$\rightarrow$ explizite Typkonvertierung

$\rightarrow$ Ändern des Registerinhalts über Dereferenzierung (* / $\rightarrow$ Operator)

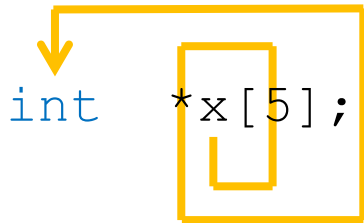
```
(*GPIOA).PIN = 0;
```

oder:

```
GPIOA->PIN = 0;
```

3.2.5 Komplexe Deklarationen

- Eine Variablendeklaration kann u.U. sehr komplex werden.
- Beispiele:

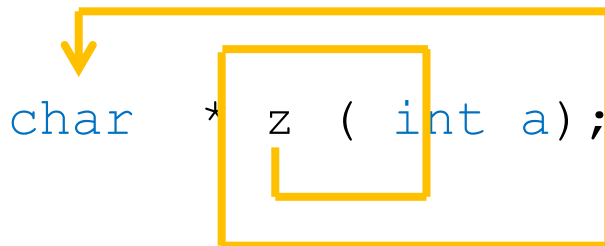


int *x[5];

- ein Array von 5
- Pointer auf
- einen Integer

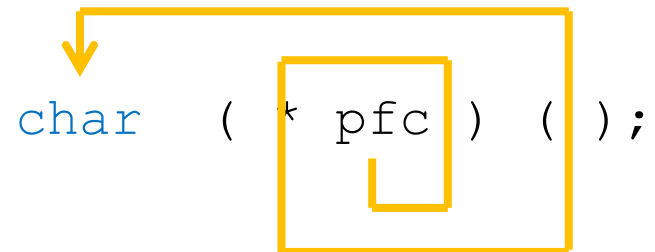
```
int a = 1, b = 2, c = 3;  
int *x[3] = { &a, &b, &c };  
  
*x[0] = 10; // a ist jetzt 10
```

„rechts / links“ – Regel: beginne mit dem Bezeichner und gehe abwechselnd nach recht/links (* hat kleinere Priorität als () und [])



char *z(int a);

Funktion, mit Rückgabe char *



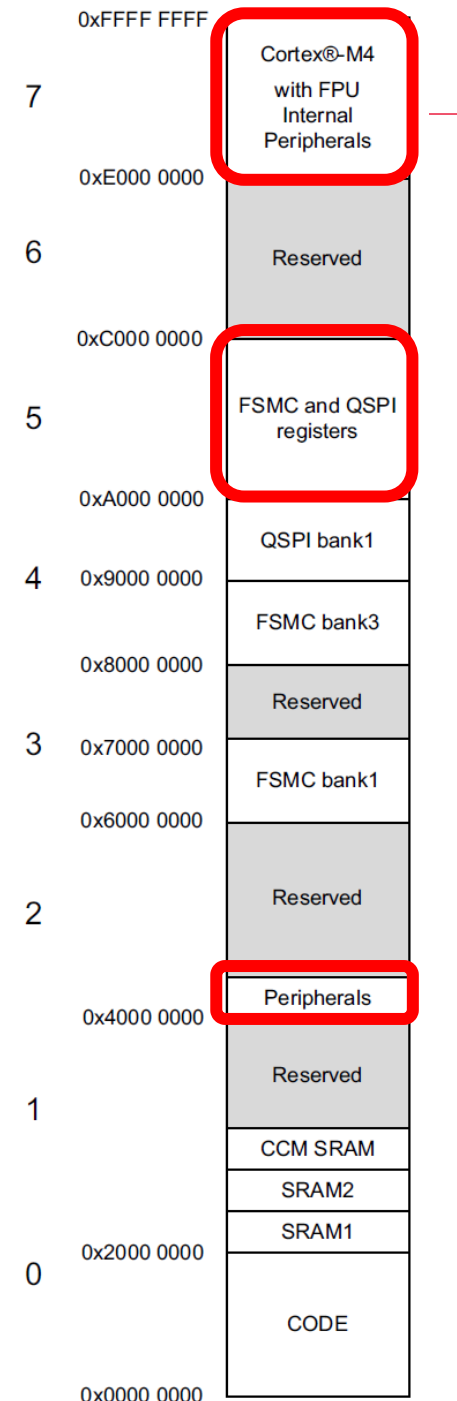
char (*pfc)();

Funktionszeiger

3.3 Bitmanipulationen

• Motivation:

- Die I/O – Register sind die Schnittstelle zur Peripherie des μ C (z.B. I/O, Timer, ADC, ...).
- Damit ist es möglich, die Peripherie zu steuern aber auch Statusinformationen zu lesen.
- Jedes I/O Register ist 32 Bit breit und ist über eine feste Adresse im Speicher ansprechbar.
- Einzelne Bits oder auch Bitgruppen haben eine definierte Bedeutung und müssen unabhängig verändert werden können.



Bitoperationen

(1) Bitweise Negation: $\sim$

$\sim 0 \rightarrow 1$
 $\sim 1 \rightarrow 0$

Beispiele:

```
unsigned char a = 0xF0;  
unsigned char b = ~a;
```

(2) bitweise AND: $\&$

$\&$	0	1
0	0	0
1	0	1

```
unsigned char a = 0xF0;  
unsigned char m = 0x11;  
unsigned char b = a&m;
```

(3) bitweise OR: $|$

$ $	0	1
0	0	1
1	1	1

```
unsigned char a = 0xF0;  
unsigned char m = 0x11;  
unsigned char b = a|m;
```

(4) bitweise XOR: $\wedge$

$\wedge$	0	1
0	0	1
1	1	0

```
unsigned char a = 0xF0;  
unsigned char m = 0x11;  
unsigned char b = a^m;
```

Wie lautet der Wert der Variablen **b** in jedem Beispiel?

- Anwendungen zu Bitoperationen in einer Embedded SW:
 - Löschen von bestimmten Bits in einem Register → AND:

```
PORTA = PORTA & ~0x80;    // PORTA &= ~0x80;
```

- Setzen von bestimmten Bits in einem Register → OR:

```
DDRA  = DDRA | 0x01;      // DDRA |= 0x01;
```

- Invertieren (*Toggeln*) von bestimmten Bits in einem Register → XOR

```
PORTA = PORTA ^ 0x10;    // PORTA ^= 0x10;
```

Hinweis: Für die Maskierung von Bits verwendet man oft eine Kombination aus Shift- und Bitoperation:

```
DDRA  = (1<<7) | (1<<6) | (1<<5) | (1<<4);
```



Bit 0 = erstes Bit!

3.4 Funktionen

- Funktionen sind die “elementaren Bausteine” eines C-Programmes.
- auch: Unterprogramme, Prozeduren
- Motivation / Vorteile:
 - **Wiederverwendbarkeit:**
Teile eines Programms, die an verschiedenen Stellen verwendet werden sollen, müssen nur einmal definiert werden.
 - **Verringerung der Komplexität**
Zerlegung eines Programmes in kleinere überschaubare Teileinheiten
(*Modularisierung, Abstraktion, “Trennung der Zuständigkeiten”*)
 - **Kapselung:**
Verbergen von Implementierungsdetails
 - Getestete Funktionen können z.B. zu Funktionsbibliotheken zusammengefasst werden.

3.4.1 Definition / Deklaration von Funktionen

- Syntax (Definition):

```
Rückgabetyp Funktionsname ( Typ Parameter, ... )  
{  
    Anweisungen  
}
```

- Funktionen werden immer global definiert (→ keine verschachtelten Definitionen möglich!).
- Kein Unterschied zwischen Prozeduren (Rückgabetyp: `void`) und Funktionen
- Funktionen sollten deklariert (bekannt gemacht) werden, bevor sie aufgerufen werden:
 - **forward-Deklaration**
(ansonsten Annahme, dass der Rückgabewert vom Typ `int` ist
→ implizite Deklaration erzeugt Compilerwarnung)

```
/* Deklaration (Funktionsprototyp) */  
void bar(int);  
  
void foo(int b) {  
    if (b<0) return;  
    bar(b-1);  
}  
  
/* Definition */  
void bar(int a) {if (a<0) return;  
    foo(a-1);  
}
```

3.4.2 Wertaustausch

(1) call by value (technisch einzige Möglichkeit in C)

- *aktuelle Parameter* werden in die *formalen Parameter* kopiert

(2) Pointer as value (call by reference, & Typmodifikator in C++)

- Zeiger wird auch call by value übergeben (→ Kopie der Adresse).
- Verwendung des * Operators zum Zugriff auf die referenzierten Variablen innerhalb der Funktion:
- Beispiel: Vertauschen von Variableninhalten

```
void swap(int *a, int *b)
{
    int tmp = *a;
    *a = *b;
    *b = tmp;
}
```

Was gibt dieses Programm aus?

```
#include <stdio.h>

void foo(int a) {
    a++;
}

int main() {
    int a=5;
    foo(a);
    printf("%d",a);
    return 0;
}
```

3.4.3 Funktionszeiger

- Verweis auf Funktion, umlenkbar auf andere Funktionen des gleichen Typs
- Funktionszeiger sind Zeiger auf eine Funktion (anstelle von Variablen)
 - Funktionszeiger zeigt auf die erste Instruktion der Funktion im Speicher
- Verwendung:
 - Dynamischer Aufruf von Funktionen in einem Scheduler
 - Identische Schnittstellen für Gerätetreiber, Callback-Funktionen

- Syntax:

*Rückgabewert (*Funktionszeigername)(Typ Parameter);*

Zeiger auf eine Funktion
mit Rückgabotyp

Name der Zeiger-
Variablen ist "FuncPtr"

Parameterliste der
referenzierten

- Beispiel für Funktionszeiger:

```
#include <stdio.h>

int f1(int a) { return a+1; }
int f2(int a) { return a+2; }
int f3(int a) { return a+3; }

int main(void) {
    int ret = 0;
    int (*fptr)(int);

    fptr = f1;  ret = (*fptr)(0);  printf("%d\n", ret);
    fptr = f2;  ret = (*fptr)(0);  printf("%d\n", ret);
    fptr = f3;  ret = (*fptr)(0);  printf("%d\n", ret);
    return ret;
}
```

3.4.4 Schnittstelle zu Assembler

- **Ganz ohne Assembler-Programmierung geht es nicht!**
- Zwei Methoden, Assembler-Code von C aus zu benutzen:
 - (1) getrennte Quellcodedateien
 - eigene Assembler-Quellcode-Dateien (Endung „.S“) mit Funktionen:
 - Funktionsprototypen (*.h Datei) in C
 - Definierte Register als Schnittstelle für Parameter / Rückgabewerte
 - (2) Inline-Assembler: Einfügen von Assembler-Instruktionen in C-Code
 - Wichtig: Register, die überschrieben werden, könnten vom Compiler genutzt werden
 - Daher: Angabe der Eingabeparameter, Ausgabeparameter und mit Werten beschriebenen Register der CPU notwendig!
 - Aber: i.d.R. nicht portierbar

3.4.4 Schnittstelle zu Assembler

Einfache Beispiele:

```
__asm {  
    MOV R0, #0  
    ADD R0, R0, #1  
    NOP  
}
```

```
int ergebnis;  
int a = 5, b = 3;  
  
__asm {  
    ADD ergebnis, a, b  
}
```

Auf das absolut notwendige einschränken. Wir verwenden im Praktikum eine Delay Funktion, welche in Assembler programmiert wurde.

- ARM Architecture Procedure Call Standard (AAPCS):
 - Regelt Aufrufkonventionen und Parameterübergabe in C-Programmen.
 - Für die **Parameterübergabe** an Funktionen werden die Register R0–R3 verwendet. Weitere Parameter werden auf dem Stack übergeben.

Parametertyp	Register
<code>_Bool, char, short, int, long, float, 32-bit pointer, 32-bit struct</code>	R0, R1, R2, R3
<code>long long, double, 64-bit struct</code>	R0R1, R1R2, R2R3

- Für **Rückgabewerte** dient R0 (32 Bit), R0 : R1 (64 Bit) oder der Stack (größer als 64 Bit).

Parametertyp	Register
<code>_Bool, char, short, int, long, float, 32-bit pointer, 32-bit struct</code>	R0
<code>long long, double, 64-bit struct</code>	R0 : R1

- ARM Architecture Procedure Call Standard (AAPCS):

- **Beispiel:**

```
int func(int a)
{
    int b;
    b = 2*a;
    return b;
}
```

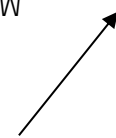


0x000002C0	4601	MOV	r1, r0
0x000002C2	0048	LSLS	r0, r1, #1
0x000002C4	4770	BX	lr

```
X = func(2);
```



0x000003E4	2002	MOVS	r0, #0x02
0x000003E6	F7FFF6B	BL.W	func
0x000003EA	4605	MOV	r5, r0
...			



Adresse: 0x000002C0

Hinweis: <https://godbolt.org/>

- Funktionsdefinition in Assembler:

- Aufruf:

main.c

```
// Deklaration
void delayms ( uint32_t msec );

...

delayms(500);

...
```

```
AREA  DELAY, CODE, READONLY
THUMB

delayms FUNCTION          ; start of function
    EXPORT delayms        ; .global delayms

    PUSH {R1}             ; store used registers
    MOV     R1, #12000     ; = 0.001 s / (4 *
                           ; 1/48000000Hz)

    MUL     R0, R0, R1

    ALIGN   4              ; following instruction to be
                           ; 32Bit aligned

loop_delay_ms
    SUB     R0, R0, #1
    CMP     R0, #0
    BNE     loop_delay_ms
    NOP

    POP     {R1}
    BX      lr

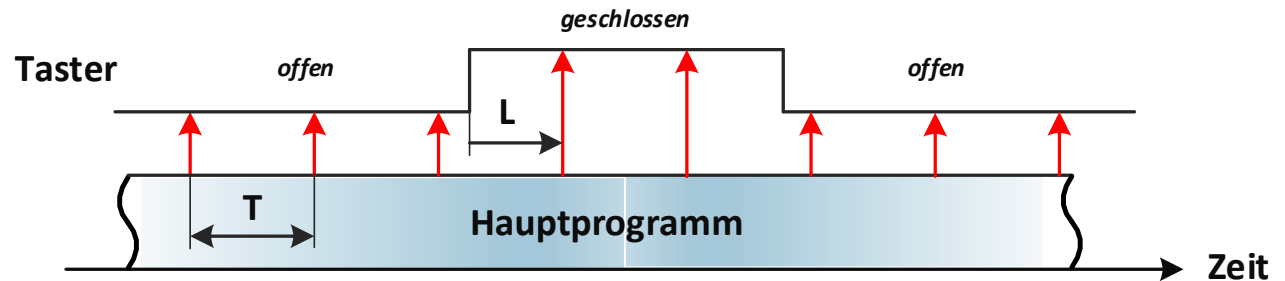
    ENDFUNC                ; end of function
END
```

3.5 Interrupts

- Embedded Systeme sind **reaktive Systeme**, d.h. sie stehen in ständiger Wechselwirkung mit der Umgebung.
 - ⇒ Reaktion auf externe Ereignisse oder Eingaben
z.B. Sensorsignale, Benutzereingaben, Kommunikation, ...
 - ⇒ Abhängig von diesen Eingaben gibt es Reaktionen und Ausgaben an die Umwelt.
- Merkmale von Interrupts:
 - Interrupts können zu **beliebigen Zeitpunkten** auftreten.
 - Interrupts **unterbrechen** das laufende Programm kurzfristig.
 - ⇒ Je kleiner die **Umschaltzeit (Interrupt Latency)** ist, desto besser ist der Mikrocontroller für zeitkritische Anwendungen geeignet.
 - Interrupts müssen **priorisiert** werden, falls mehrere gleichzeitig auftreten können.
 - Interrupts sollten per Software **maskierbar** (abschaltbar) sein, d.h. Programmteile müssen sich ggf. gegen Interrupts schützen können.

- Ein μ Controller kann ein Ereignis auf zwei Arten erkennen:

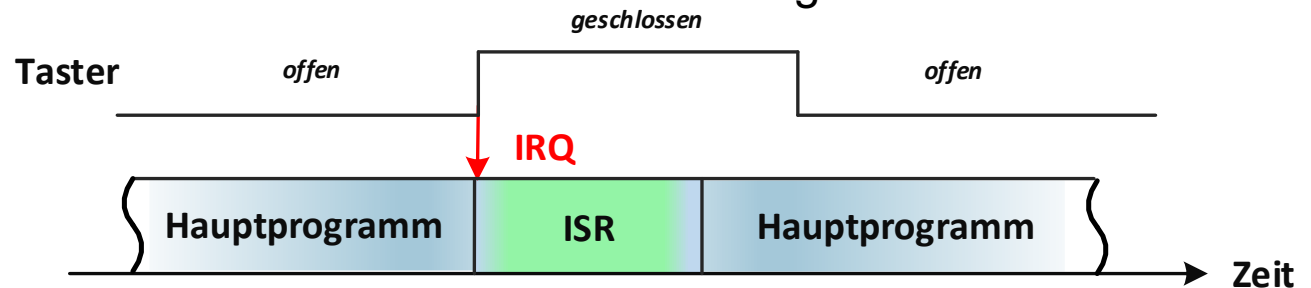
1. Polling: Die Software frägt zyklisch einen Zustand ab.



Nachteil: Rechenzeit / Latenzzeit (= Antwort auf Statusänderung)

2. Interrupt:

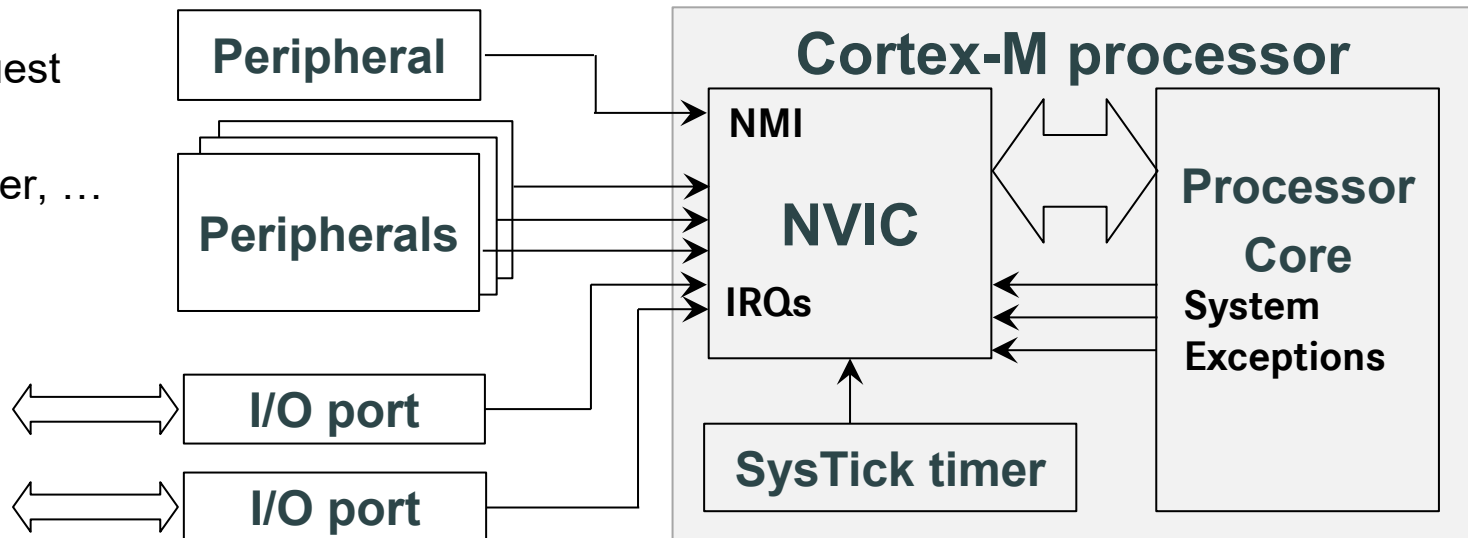
Anders als zyklisch den Zustand abzufragen, reagiert ein Interrupt sofort auf die Zustandsänderung. Der Interrupt „fordert Rechenzeit“ von der CPU an und unterbricht so den aktuellen Programmlauf.



- NVIC: (*Nested Vectored Interrupt Controller*)
nennt man die HW eines Mikrocontrollers (hier: Teil des Cortex Prozessors),
welche die Abarbeitung eines oder mehrerer Interrupts steuert.
 - unterstützt max. 240 Interrupts (→ STM32G474: 102)
 - max. 256 Prioritätsebenen (→ STM32G474 : 16)
 - Unterscheidung: Exceptions, Non-Maskable-Interrupts (NMI), SysTick, Maskable Interrupts (IRQ), externe Interrupts

IRQ =
Interrupt **R**equ**R**est

z.B. ADC, Timer, ...



• Interruptvektortabelle:

- Jeder Interruptquelle ist ein Vektor zugeordnet.
- Vektor = Startadresse des Interrupt-Handlers
- Assembler:

startup.s

```
; Vector table start
        AREA      RESET, DATA, READONLY
        THUMB

__Vectors DCD __initial_sp  ; Stackpointer
          DCD Reset_Handler ; Initialer PC
          DCD NMI_Handler
          ...
```

**Speicher-
adresse**

0x0000004C
0x00000048
0x00000044
0x00000040
0x0000003C
0x00000038
0x00000034
0x00000030
0x0000002C
0x00000028
0x00000024
0x00000020
0x0000001C
0x00000018
0x00000014
0x00000010
0x0000000C
0x00000008
0x00000004
0x00000000

	1	
	1	
Interrupt#3 vector	1	
Interrupt#2 vector	1	
Interrupt#1 vector	1	
Interrupt#0 vector	1	
SysTick vector	1	
PendSV vector	1	
Not used		
Debug Monitor vector	1	
SVC vector	1	
Not used		
Not used		
Not used		
Not used		
Usage Fault vector	1	
Bus Fault vector	1	
MemManage vector	1	
HardFault vector	1	
NMI vector	1	
Reset vector	1	
MSP initial value		

**Exception
Number**

19
18
17
16
15
14
13
12
11
10
9
8
7
6
5
4
3
2
1
0

Note : LSB of each vector must be set to 1 to indicate Thumb state

- Abarbeitung von Interrupts:

Das Interrupt-Handling ist HW-gesteuert (micro-coded in Interrupt Controller) und beansprucht keinen Overhead in Form von Software.

- Aktionen bei Eintritt eines Interrupts:

(1) **Stacking** = Sichern von Registern auf dem aktuellen Stack (PSP/MSP)

(2) Laden des **Interrupt Vektors**
→ Startadresse des Interrupt-Handlers

(3) Änderungen von **NVIC / Core Register**

– $SP = oldSP - 32$

– $PC = \text{Startadresse der ISR}$

– $LR = EXC_RETURN$ (Bits 3..0: Kennung für Rücksprung)

1..10001: Return to Handler Mode, use MSP

1..11001: Return to Thread Mode, use MSP

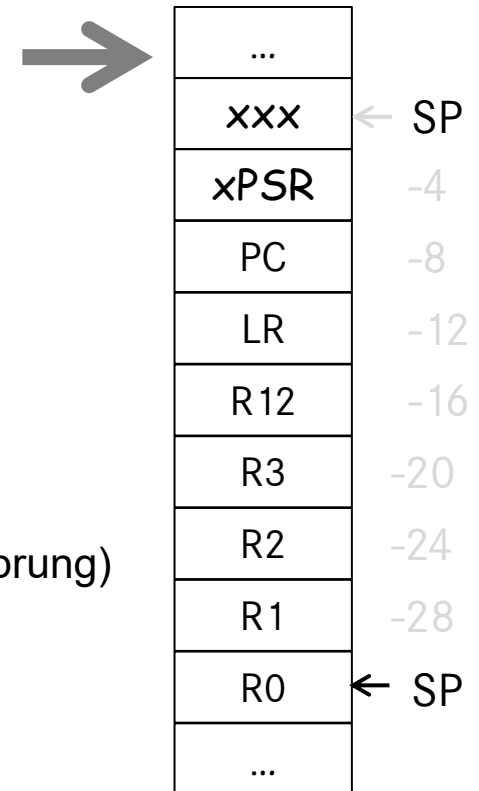
1..11101: Return to Thread Mode, use PSP

– $IPSR = \text{Nummer der aktuellen ISR}$

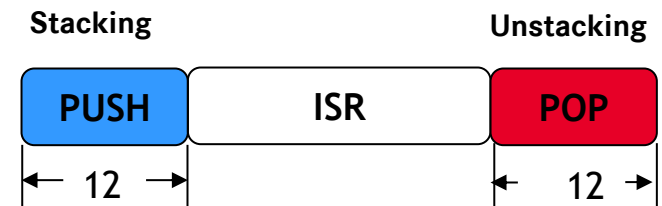
– Update der NVIC-Register: Pending Bit = 0, Active Bit = 1

Das LR enthält
nicht die
Rücksprung-
adresse!

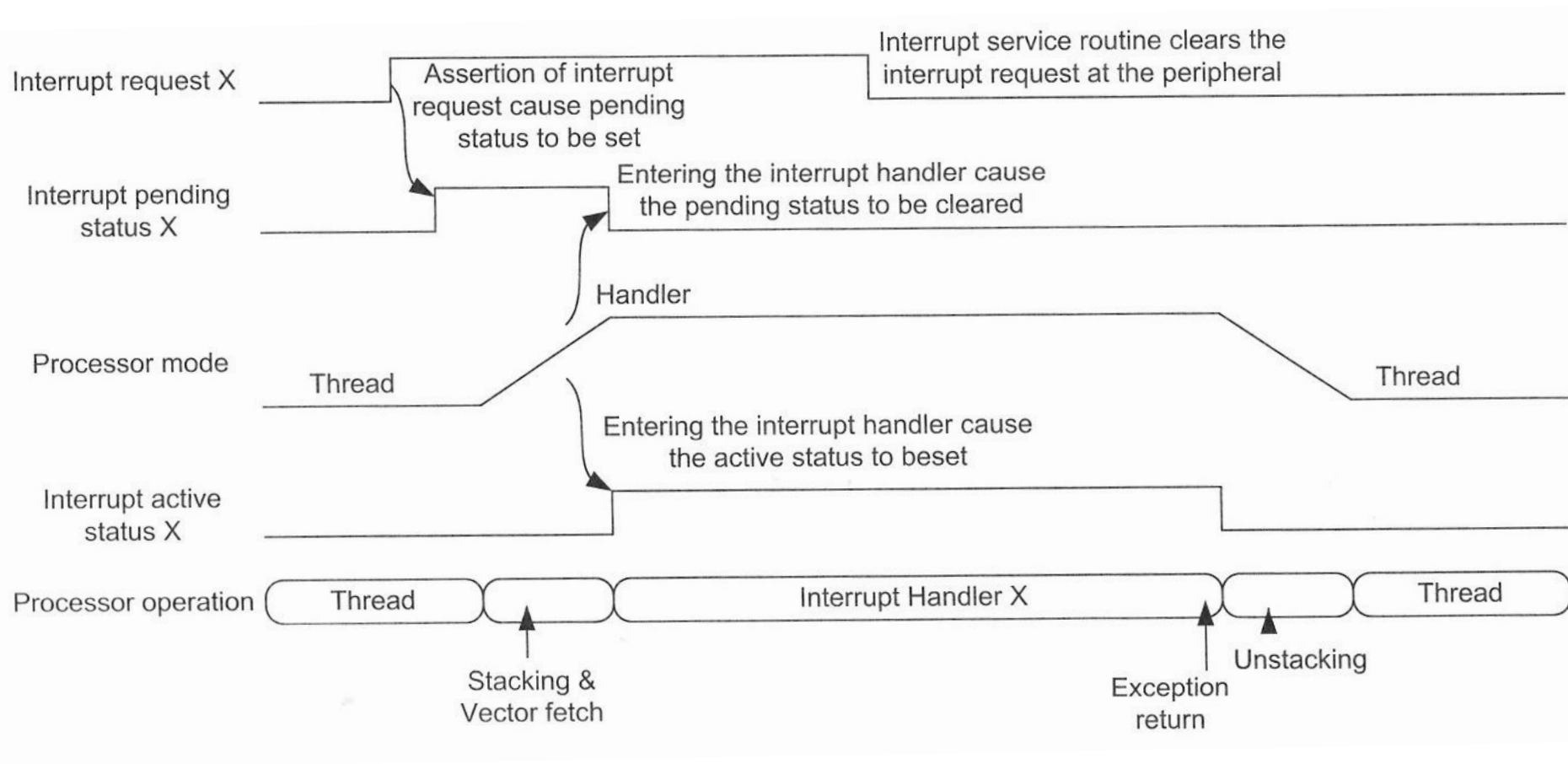
(PUSH/Stacking:
12 Zyklen)



- Ausführung eines Interrupts:
 - User Code in Interrupt Handler (→ C-Funktion)
 - falls Assembler-Funktion, ggf. Register R4 – R11 sichern/wiederherstellen
 - Interrupt Handler so kurz wie möglich
 - kann durch einen Interrupt höherer Priorität verdrängt werden
- Beendigung eines Interrupts:
 - Rücksprung erfolgt durch Beendigung der ISR (auf Assemblerebene z.B. durch `BX LR, C` mit `{ }`)
 - (1) **Unstacking** = die vorher gesicherten Registerinhalte werden zurückgeladen (POP: 12 Zyklen)
 - (2) Rücksprungadresse aus dem zuvor gesicherten Stack
→ Unstacking wird angestoßen durch Laden des `LR` (`EXC_RETURN`) in `PC`
 - (3) Update der NVIC-Register: Active Bit = 0



- Zusammenfassung:



- Programmierung der zugehörigen ISR (Interrupt Service Routine):
 - Es existieren vordefinierte Bezeichner (→ Startup-File) für die Interrupt Service Routinen:
 - **Grundregeln:**
 - ISRs so schnell wie möglich wieder verlassen
 - Innerhalb einer ISR
 - Keine blockierenden Operationen (z.B. Aufruf von `scanf()`, lange Warteschleifen, ...)
 - Falls in Assembler programmiert wird: ggf. Register (R4-R11) sichern/restaurieren
 - Nicht verwendete Interrupts deaktivieren oder an Standardhandler weiterleiten
 - Auf Datenkonsistenz bei Verwendung globaler Variablen achten (Interrupt ggf. vor schreibenden Zugriff sperren und nach Zugriff freigeben)

```
void NMI_Handler           (void) ;
void HardFault_Handler     (void) ;
void MemManage_Handler     (void) ;
void BusFault_Handler      (void) ;
void UsageFault_Handler    (void) ;
void SVC_Handler           (void) ;
void DebugMon_Handler      (void) ;
void PendSV_Handler        (void) ;
void SysTick_Handler       (void) ;
void WWDG_IRQHandler        (void) ;
void PVD_PVM_IRQHandler    (void) ;
void RTC_TAMP_LSECSS_IRQHandler (void) ;
void RTC_WKUP_IRQHandler   (void) ;
void FLASH_IRQHandler       (void) ;
void RCC_IRQHandler         (void) ;
void EXTI0_IRQHandler       (void) ;
void EXTI1_IRQHandler       (void) ;
void EXTI2_IRQHandler       (void) ;
void EXTI3_IRQHandler       (void) ;
void EXTI4_IRQHandler       (void) ;
void DMA1_Channel1_IRQHandler (void) ;
...
```

(2) Programmierung der zugehörigen ISR:

interrupt.c

```
extern uint8_t Direction;
...
void EXTI0_IRQHandler (void)
{
    EXTI->PR1 = 1;
    Direction ^= 0x01;
}
```

→ Warum sollte hier die globale Variable `Direction` **volatile** deklariert werden ?

main.c

```
#include <stdio.h>
#include <stdlib.h>
#include <stm32g474xx.h>
#include "leds.h"
#include "delay.h"

volatile uint8_t Direction;

void main(void)
{
    ...
    while(1)
    {
        switch(Direction)
        {
            case 0:
                ...
                break;
            case 1:
                ...
                break;
            default:
                break;
        }
        SetLEDs(Leds);
        delayms(1000);
    }
}
```

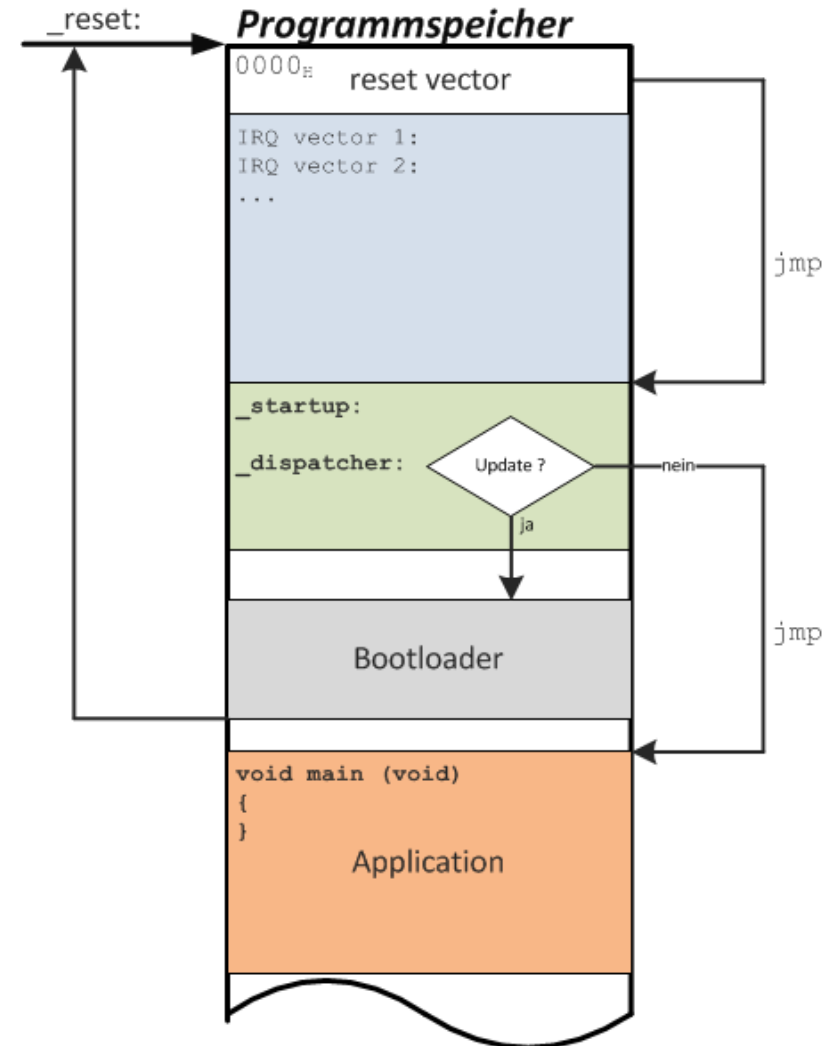
4. Embedded Software

a) Systemstart

- Nach einem **Reset** liest ein Controller als erstes die Instruktion im Resetvektor (meist Adresse `0x00000000`).
- Hier muss ein gültiger Befehl liegen! (i.d.R. Sprungbefehl zum sog. Bootcode)

Bootcode:

- Initialisierung des Speichers
- Initialisierung der Peripherie
- am Ende Sprung in die `main()` - Routine der Applikation

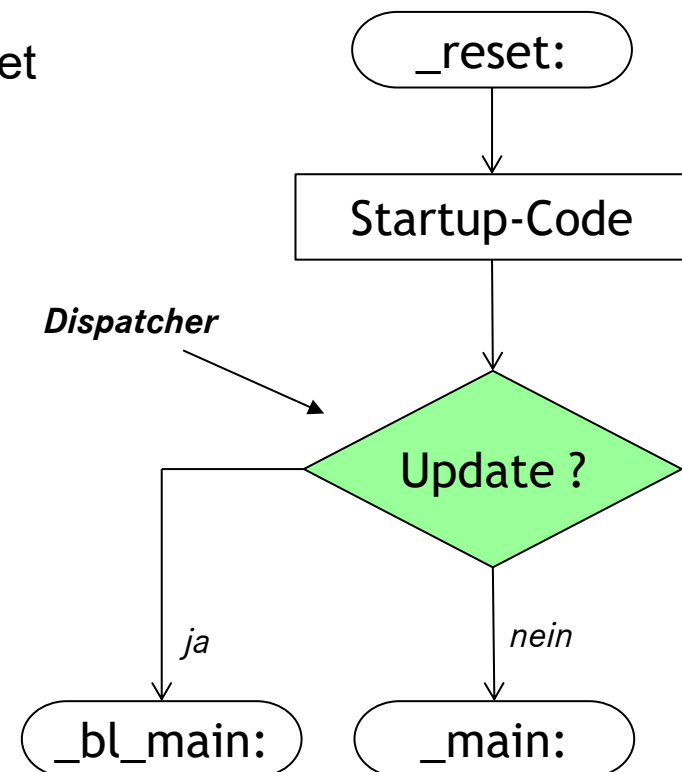


b) Bootcode

- Bootcode muss ausgeführt werden, bevor die Laufzeitumgebung einer höheren Programmiersprache initialisiert ist.
 - ⇒ **Bootcode ist immer in Assembler geschrieben!**
- Bootcode ist abhängig von der verwendeten HW (CPU, Speicher, Peripherie) sowie dem eingesetzten Compiler.
- Gerüst des Bootcodes gehört zum Lieferumfang eines Compilers (**Start-up Code**).
 - Der Applikationsentwickler nimmt die spezifischen Anpassungen an „seine“ Hardware vor
 - bei updatefähigen Steuergeräten:
 - ⇒ Bootcode darf nicht gelöscht werden
 - ⇒ zusätzliche Funktionalität: Software - Download

c) Bootloader

- Für Embedded Systeme (ES) gilt meist die Anforderung an ein **Software – Update** im Feld (bei Automotive → meist Werkstatt).
- Dazu wird das ES in einem speziellen **Update – Mode** gestartet:
- Der sog. **Dispatcher** entscheidet während des Systemstart, ob ein Update stattfindet.
⇒ Parameter, Nutzereingabe während Reset
- Der **Bootloader** ersetzt im Flash-Speicher den Programmcode der Applikation:
 1. **Löschen** der (alten) Applikation
 2. **Download** über Diagnoseschnittstelle (z.B. CAN)
 3. **Schreiben** der neuen Applikation
 4. **Verifikation** (Überprüfung) der programmierten Applikation



d) Applikation

- Die **Applikations – Software** realisiert die eigentliche Funktion des Embedded Systems.
- Der **Einsprungpunkt** der Applikation ist in C die **main()** – Routine.
- Ohne Betriebssystem (*engl. operating system – OS*) werden die Funktionen nacheinander in einer Endlosschleife ausgeführt („*Scheduling*“).
- zunehmender Einsatz von **Echtzeitbetriebssystemen** (z.B. OSEK, freeRTOS, ...) ⇒ (Haupt-) Funktionen → Tasks

```
int main( void )
{
    /* Initialisierungen */

    /* Endlos - Schleife */
    while( 1 )
    {
        /* Applikations Code */
    }

    /* */
    return 0;
}
```

5. Tipps und Tricks

5.1 Häufige Fehler

(1) nicht-initialisierte Variablen

```
void DelayLoop(void)
{
    char a;
    while (a<100)
    {
        a++;
    }
}
```

Wie oft wird die Schleife ausgeführt?

(2) implizite Typumwandlung (*Promotion*) Konvertierung von Datentypen

```
unsigned int a = 6 ;
int b = -20;
if ( a + b > 6)
    ExecuteBig();
else
    ExecuteSmall();
```

Welcher Zweig wird ausgeführt?

(3) Zuweisungsoperator statt Vergleichsoperator

- meist Flüchtigkeitsfehler

```
if ( a = 1 )  
{  
  ...  
}
```

statt

```
if ( a == 1 )  
{  
  ...  
}
```

- Daher verwendet man bei Vergleichen an erster Stelle den konstanten Ausdruck:

```
if ( 1 == a )  
{  
  ...  
}
```

⇒ `if ( 1 = a )`
führt hier zu einem Compilerfehler.

(4) Verwechseln der Reihenfolge von pre– und post – Inkrement/Dekrement Operator

- Beispiel: Durchlaufen aller Zeichen eines Strings

```
char a[] = "hallo";  
char* pa = a;  
while ( *++pa != '\0')  
{  
    ...  
}
```

Was läuft hier falsch?

- Je nach Stellung des Inkrement/Dekrement Operators sind zwei Fälle zu unterscheiden:

→ Reihenfolge der Bewertung und Operation

- postfix: *Variable++*
(Bewertung → Erhöhung)
- prefix: *++Variable*
(Erhöhung → Bewertung)

(5) Mehrfache Verwendung von inkrementierten bzw. dekrementierten Variablen

- Beispiel:

```
int i = 2;  
a = ++i * 2 + i * 3;
```

Was ist hier das Problem?

- Inkrement / Dekrement Operatoren verändern den Wert der Variablen.

(6) Überlauf eines Arrays

- Beispiel:

```
int i;  
char a[] = "hallo";  
for(i=0; i<20; i++)  
{  
    Execute( a[i] );  
}
```

(7) kein 'if' ohne 'else'

- 'else' kann wegfallen, wenn es nur zwei Wertebereich gibt:

```
if ( c < 10 )    c = 10;
```

- kein 'else' ist gefährlich wenn mehrere mögliche Bereiche ignoriert werden:

```
if ( (c == 10) || (c == 20) )
```

- vermeide „Löcher“ in 'else if' Ketten:

```
if ( (c > 10) && (c < 20) )  
....  
else if ( (c > 20) && (c < 30) )  
....
```

Besser:

```
if(...)  
{  
}  
else  
{  
    if(...)  
    {  
    }  
}
```

(8) kein 'switch' ohne 'default' & kein 'case' ohne 'break'

- Um unbehandelte Fälle zu vermeiden, sollte 'default' in keiner 'switch' Anweisung fehlen.

Beispiel:

```
int i;  
....  
switch ( i )  
{  
    case 0:  
        ....  
        break;  
  
    case 1:  
        ....  
  
    case 2:  
        ....  
        break;  
  
    default:  
        ....  
        break;  
}
```

häufiger Fehler
→ fehlendes **break**

Ausnahme:

wenn für zwei Fälle die
gleiche Behandlung
gefordert ist
→ *fall-through*

***Besonderheiten
kommentieren!***

(9) Verwendung von Vergleichsoperatoren mit strings

```
if ( s == "end" )  
{  
  ...  
}
```

statt

```
if ( 0 == strcmp(s, "end") )  
{  
  ...  
}
```

übrigens: Was ist der Unterschied?

```
char *s1 = "abcdefg";  
char s2[] = "abcdefg";
```

Hinweis:
in C++ mit
std::string ist
der ==
Operator
definiert.

(10) Verwendung von floats und Divisions

- Eigentlich kein Fehler, aber bei μC ohne Hardwareunterstützung für Gleitkommazahlen und Divisionen (z.B. AVR) müssen diese Operationen mittels SW – Bibliotheken realisiert werden.

⇒ Laufzeiten !

5.2 Weitere Tipps

(11) Code-Dokumentation / lesbarer Code

- Negativbeispiele: Was machen die Funktionen?

```
void sc( char *s, char *d )  
{  
    while ( *d++ = *s++ );  
}
```

- geeignete Einrückung
- verwende aussagekräftige Bezeichner für Variablen und Funktionen
z.B. Ein Funktionsname soll ausdrücken, was die Funktion macht
(Verb + Objekt)
- aussagekräftige Kommentare
- Tool zur Codedokumentation (z.B. *Doxygen*)

```
int is( char *s, char c )  
{  
    char *t = s;  
    while ( *t )  
        if ( *t++ == c )  
            return t - s - 1;  
    return -1;  
}
```

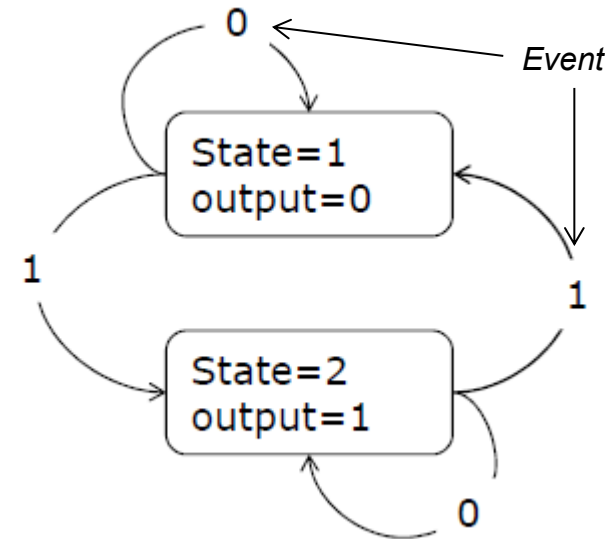
(11) Zustandsautomaten

- Zustandsautomaten (*state machine*) werden in *Embedded Systemen* oft dazu verwendet, um ein Verhalten in Abhängigkeit eines Systemzustandes zu realisieren.
- Wie werden State Machines in C codiert?

⇒ zwei Möglichkeiten

a) switch - case

```
switch( State )
{
    case 1:
        output = 0;
        if( 1 == event ) State = 2;
        break;
    case 2:
        output = 1;
        if( 1 == event ) State = 1;
        break;
    default :
        break;
}
```

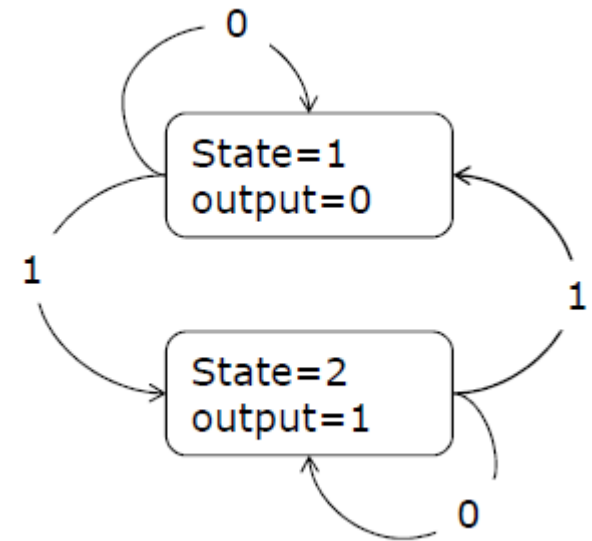


⇒ wird bei sehr vielen Zuständen lang und unübersichtlich!

b) Tabelle

State \ Event	neuer Zustand	
	0	1
1	State = 1 / output = 0	State = 2 / output = 1
2	State = 2 / output = 1	State = 1 / output = 0

Aktion



- Vorteile gegenüber der `switch-case` Lösung:
 - einfache Überprüfung, ob alle Zustände betrachtet wurden
 - Trennung zwischen Event und Aktion

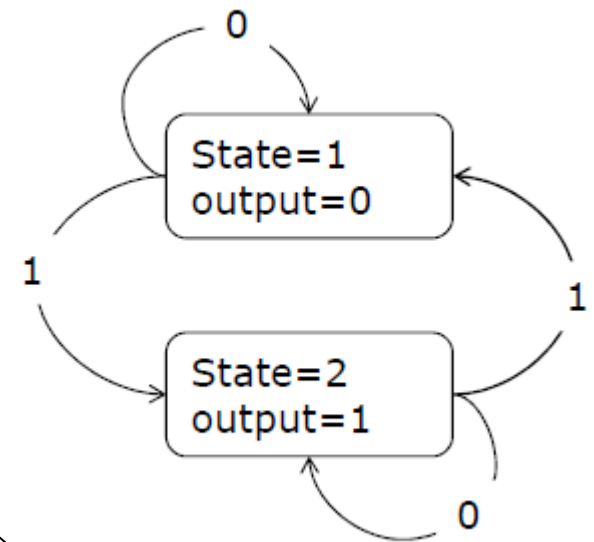
• Realisierung einer tabellengestützten State Machine

```
typedef struct {
    uint8_t    newstate;
    void      (*fct_action)(void);
} tbl_entry;

void fct_statel(void)
{
    output = 0;
}
void fct_state2(void)
{
    output = 1;
}

const tbl_entry table[2][2] =
    { { {1,fct_statel},{2,fct_state2} },
      { {2,fct_state2},{1,fct_statel} } };

void StateMachine (uint8_t evt)
{
    static uint8_t  State = 1;
    ...
    table[State-1][evt].fct_action();
    State = table[State-1][evt].newstate;
}
```



*Struktur eines Tabelleneintrages
(hier: Funktionszeiger)*

*Funktion je Aktion bei Wechsel
in einen neuen Zustand*

*2-dimensionale Tabelle
(Zustand/Event)*

• Essentielle Unterschiede

Aspekt	Desktop	Embedded
Heap / malloc	<pre>int *p = malloc(4*sizeof(int));</pre> OS verwaltet Speicher	<pre>static int buf[4];</pre> Kein malloc Statische Allokation Deterministische Zeit
Standard I/O	<pre>printf("Hallo");</pre> stdout existiert OS leitet an Terminal	<pre>USART1->TDR = 'H';</pre> Kein stdout Direkt auf Register Retargeting nötig
Hardware-Zugriff	<pre>fd = open("/dev/gpio", O_RDWR);</pre> Syscalls ans OS Treiber abstrahiert	<pre>GPIOA->ODR = (1 << 5);</pre> Direkte Registeradresse Kein Treiber, kein OS
volatile	Selten verwendet Signal-Handler, Threading	<pre>volatile uint32_t *reg = 0x40020000;</pre> Verhindert Wegoptimierung
Startup-Code	<pre>int main() {</pre> Loader lädt ELF libc init transparent Entwickler sieht nichts	<pre>Reset_Handler:</pre> SP setzen, .data kopieren .bss nullen, Clocks init Dann erst main()
Linker-Script	Meist irrelevant OS virtualisiert Speicher Standard-Layout	<pre>FLASH : 0x08000000 RAM : 0x20000000</pre> Explizite Speichermap Entwickler definiert alles